



**Faculty Minden**

**Masterthesis**

Computer Science

2026

**Design of an extendible Intermediate Representation for  
Multi-Language Game-Based Programming Education**

Lasse Foster

Matrikel-Nr.: 50049857

First Supervisor: Prof. Dr.-Ing. Carsten Gips  
Second Supervisor: Dipl.-Inf. Birgit Christina George

# Contents

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| <b>List of Figures</b>                                            | <b>i</b>   |
| <b>List of Tables</b>                                             | <b>ii</b>  |
| <b>List of Abbreviations</b>                                      | <b>ii</b>  |
| <b>Overview of Tools Used</b>                                     | <b>iii</b> |
| <b>1 Introduction</b>                                             | <b>1</b>   |
| <b>2 Overview of Educational Programming</b>                      | <b>3</b>   |
| 2.1 History and Motivation . . . . .                              | 3          |
| 2.1.1 LOGO . . . . .                                              | 3          |
| 2.1.2 Karel the Robot . . . . .                                   | 4          |
| 2.1.3 Scratch . . . . .                                           | 4          |
| 2.1.4 Blockly . . . . .                                           | 4          |
| 2.1.5 Robot Karol Online . . . . .                                | 5          |
| 2.2 Robot Karol Online: Technological Foundation . . . . .        | 6          |
| <b>3 The Probus Project &amp; Current Limitations</b>             | <b>8</b>   |
| 3.1 Limitations . . . . .                                         | 8          |
| 3.2 Target . . . . .                                              | 10         |
| <b>4 Compiler Infrastructure and Intermediate Representations</b> | <b>11</b>  |
| 4.1 Linear and Graphical IRs . . . . .                            | 11         |
| 4.2 Polish Notation . . . . .                                     | 12         |
| 4.3 Three-Address Code . . . . .                                  | 12         |
| 4.4 Static Single Assignment . . . . .                            | 13         |
| 4.5 Control Flow Graph . . . . .                                  | 14         |
| 4.6 Data Flow Graphs and Def-Use Chains . . . . .                 | 15         |
| 4.7 Low-Level, Flat Representations . . . . .                     | 16         |
| 4.8 Multi-Level, Hierarchical Representations . . . . .           | 17         |
| <b>5 DGIR Methodology and Architectural Evolution</b>             | <b>20</b>  |
| 5.1 Methodological Framework . . . . .                            | 20         |
| 5.2 Evaluation Design . . . . .                                   | 20         |
| 5.3 Threats to Validity . . . . .                                 | 20         |
| 5.3.1 Construct validity. . . . .                                 | 21         |
| 5.3.2 Internal validity. . . . .                                  | 21         |
| 5.3.3 External validity. . . . .                                  | 21         |
| 5.4 Architectural Design Evolution . . . . .                      | 21         |
| 5.4.1 Extensions . . . . .                                        | 21         |
| 5.4.2 Values . . . . .                                            | 22         |
| 5.4.3 Types . . . . .                                             | 22         |
| 5.4.4 Operations . . . . .                                        | 24         |
| 5.4.5 Attributes . . . . .                                        | 26         |
| 5.4.6 Operation Traits . . . . .                                  | 26         |
| <b>6 Results</b>                                                  | <b>28</b>  |
| 6.1 Final DGIR Artifact . . . . .                                 | 28         |
| 6.1.1 JSON IR Form . . . . .                                      | 28         |
| 6.1.2 Textual IR Form . . . . .                                   | 31         |
| 6.1.3 Validation and Analysis . . . . .                           | 31         |
| 6.1.4 Implemented Extension Surface . . . . .                     | 32         |
| 6.1.5 Interpreter Workflow . . . . .                              | 34         |
| 6.2 API Complexity . . . . .                                      | 34         |
| 6.2.1 Dialect . . . . .                                           | 35         |
| 6.2.2 Op . . . . .                                                | 35         |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| 6.2.3    | Type . . . . .                                 | 36        |
| 6.2.4    | Attribute . . . . .                            | 36        |
| 6.2.5    | OpTrait . . . . .                              | 37        |
| 6.3      | IR Simplicity . . . . .                        | 37        |
| 6.3.1    | Regularity . . . . .                           | 38        |
| 6.4      | Extensibility in Practice . . . . .            | 39        |
| 6.5      | Design Trade-offs and Non-goals . . . . .      | 40        |
| 6.6      | Summary of Findings . . . . .                  | 41        |
| <b>7</b> | <b>Discussion</b>                              | <b>42</b> |
| 7.1      | Strengths of the Final Design . . . . .        | 42        |
| 7.2      | Limitations of the Non-SSA Choice . . . . .    | 42        |
| 7.3      | Complexity Versus Expressivity . . . . .       | 43        |
| 7.4      | Automated Implementations . . . . .            | 43        |
| 7.5      | Conversion and Representation Limits . . . . . | 43        |
| 7.6      | Overall Assessment and Implications . . . . .  | 43        |
| <b>8</b> | <b>Conclusion</b>                              | <b>44</b> |
| 8.1      | Evaluation Summary . . . . .                   | 44        |
| <b>9</b> | <b>Future Work</b>                             | <b>46</b> |
| 9.1      | Transformations and Optimizations . . . . .    | 46        |
| 9.2      | SSA Support . . . . .                          | 46        |
| 9.3      | Runtime Polymorphism . . . . .                 | 46        |
| 9.4      | Textual Support . . . . .                      | 47        |
| 9.5      | Automated Implementations . . . . .            | 47        |
| 9.6      | Type System . . . . .                          | 48        |
| <b>A</b> | <b>Appendix</b>                                | <b>51</b> |

## List of Figures

|   |                                                                                                                                                                                            |    |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1 | Scratch 3.0 Programming Interface . . . . .                                                                                                                                                | 5  |
| 2 | Robot Karol Online Programming Environment[18] . . . . .                                                                                                                                   | 5  |
| 3 | Robot Karol Online Blockly Code . . . . .                                                                                                                                                  | 6  |
| 4 | The execution-flow from frontend (in this case Blockly) to game. . . . .                                                                                                                   | 8  |
| 5 | "A typical compiler structure split into phases."[31, p. 2] . . . . .                                                                                                                      | 11 |
| 6 | SSA CFG with value propagation guided by dominance rules and $\phi$ -function. Note: $x$ is defined in the first basic block and visible in the last without explicit propagation. . . . . | 13 |
| 7 | Lowering steps to go from Listing 17 to Listing 18 . . . . .                                                                                                                               | 18 |
| 8 | Parametric function type with custom syntax. . . . .                                                                                                                                       | 24 |
| 9 | CFG for DGIR program in Listing 38. . . . .                                                                                                                                                | 33 |

## List of Listings

|    |                                                                                                                                     |    |
|----|-------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Simple LOGO program that prints random sentences[30, p. 5]. . . . .                                                                 | 3  |
| 2  | The output of the program Listing 1[30, p. 5]. . . . .                                                                              | 4  |
| 3  | Simple Robot Karol program[8]. . . . .                                                                                              | 4  |
| 4  | Equivalent Robot Karol DSL code to Fig. 3. . . . .                                                                                  | 6  |
| 5  | Equivalent Java code to Fig. 3. . . . .                                                                                             | 6  |
| 6  | Equivalent Java code to Fig. 3. . . . .                                                                                             | 6  |
| 7  | Op codes used by RKO's bytecode. . . . .                                                                                            | 7  |
| 8  | Infinite counting loop in LLVM IR. . . . .                                                                                          | 13 |
| 9  | Non-SSA implementation of Listing 8 . . . . .                                                                                       | 14 |
| 10 | Same loop as Listing 8, translated to MLIR. . . . .                                                                                 | 14 |
| 11 | Same loop as Listing 8, translated to textual Bril in SSA form. . . . .                                                             | 14 |
| 12 | Linear representation of CFG in Fig. 6 . . . . .                                                                                    | 15 |
| 13 | Splitting region before constructing CFG so that exit rule holds. . . . .                                                           | 15 |
| 14 | Bril program adding 1 and 2, before printing the result[29]. . . . .                                                                | 16 |
| 15 | A simple factorial program written in Bril, give in its textual representation(left)[29] and its LLVM IR equivalent(right). . . . . | 17 |
| 16 | Generic MLIR operation syntax. . . . .                                                                                              | 18 |
| 17 | High level MLIR code for Listing 15 . . . . .                                                                                       | 19 |
| 18 | Listing 17 lowered to LLVM dialect before translating to Listing 15 . . . . .                                                       | 19 |
| 19 | DGIR identifier syntax. . . . .                                                                                                     | 21 |
| 20 | MLIR loop example illustrating SSA accumulator handling via iter args. . . . .                                                      | 22 |
| 21 | TypeDetails record from current source code. . . . .                                                                                | 24 |
| 22 | Type syntax for DGIR types. . . . .                                                                                                 | 24 |
| 23 | Contents of the Operation class in DGIR. . . . .                                                                                    | 25 |
| 24 | OperationDetails record from current source code. . . . .                                                                           | 25 |
| 25 | Contents of the Op class in DGIR. . . . .                                                                                           | 26 |
| 26 | Implementation of the IHasResult trait promising the existence of a result value. . . . .                                           | 27 |
| 27 | DGIR JSON operation syntax. . . . .                                                                                                 | 28 |
| 28 | Example function operation serialized to JSON . . . . .                                                                             | 28 |
| 29 | DGIR JSON Attribute syntax. . . . .                                                                                                 | 29 |
| 30 | DGIR JSON NamedAttribute syntax. . . . .                                                                                            | 29 |
| 31 | DGIR JSON IntegerAttribute. . . . .                                                                                                 | 29 |
| 32 | DGIR JSON TypeAttribute. . . . .                                                                                                    | 29 |
| 33 | DGIR JSON Value syntax. . . . .                                                                                                     | 30 |
| 34 | DGIR JSON Value usage example. . . . .                                                                                              | 30 |
| 35 | DGIR JSON Region syntax. . . . .                                                                                                    | 30 |
| 36 | DGIR JSON Block syntax. . . . .                                                                                                     | 30 |
| 37 | A simple do while loop program implemented in Java. . . . .                                                                         | 31 |
| 38 | Java program from Listing 37 compiled to DGIR textual form. . . . .                                                                 | 32 |
| 39 | Interpreter actions for the DGIR interpreter. . . . .                                                                               | 34 |
| 40 | Exemplary implementation of the arithmetic-dialect class. . . . .                                                                   | 35 |
| 41 | Type instance uniquing example. . . . .                                                                                             | 36 |
| 42 | Implementation of string attribute descriptor. . . . .                                                                              | 37 |

|    |                                                                                                                |    |
|----|----------------------------------------------------------------------------------------------------------------|----|
| 43 | Implementation of string attribute. . . . .                                                                    | 37 |
| 44 | Implementation of the single block OpTrait. . . . .                                                            | 38 |
| 45 | Simple conditional program written in C. . . . .                                                               | 38 |
| 46 | A simple conditional program in Bril (left) and DGIR (right) textual form. . . . .                             | 39 |
| 47 | Abstracted DGIR program from Listing 46 . . . . .                                                              | 39 |
| 48 | MLIR form of program in Listing 46. Generalized printer(left) and customized(right). . . . .                   | 40 |
| 49 | Bril program using SSA form[28]. . . . .                                                                       | 42 |
| 50 | Potential SSA syntax for DGIR adapted from Bril (Listing 49). . . . .                                          | 46 |
| 51 | Possible implementation of array type serialization using builder pattern, simplifying Listing 52 .            | 47 |
| 52 | Current implementation of array type serialization producing outputs like <i>mem.array&lt;int32, 4&gt;</i> . . | 47 |
| 53 | Approximate BNF grammar of textual DGIR programs. . . . .                                                      | 51 |
| 54 | Bril JSON IR form of Listing 46 (shortened). . . . .                                                           | 52 |
| 55 | DGIR JSON IR form of Listing 46 (shortened). . . . .                                                           | 52 |
| 56 | Iterative reaching-definitions algorithm used by DGIR during static analysis. . . . .                          | 53 |
| 57 | Basic switch statement in Java. . . . .                                                                        | 54 |
| 58 | Switch statement compiled to DGIR structured control flow. . . . .                                             | 54 |
| 59 | Implementation of function type in DGIR. . . . .                                                               | 55 |
| 60 | Table-gen declaration of MLIR's <b>ConstantOp</b> . . . . .                                                    | 56 |
| 61 | MLIR's <b>ConstantOp</b> class declaration generated by table-gen(Listing 60). . . . .                         | 57 |
| 62 | DGIR's <b>ConstantOp</b> implementation. . . . .                                                               | 58 |

## List of Tables

|   |                                                                             |    |
|---|-----------------------------------------------------------------------------|----|
| 1 | "Classification of IRs According to the Taxonomy [...]"[31, p. 5] . . . . . | 12 |
|---|-----------------------------------------------------------------------------|----|

## List of Abbreviations

|             |                                     |
|-------------|-------------------------------------|
| <b>GUI</b>  | Graphical User Interface            |
| <b>API</b>  | Application Programming Interface   |
| <b>IR</b>   | Intermediate Representation         |
| <b>IPC</b>  | Inter Process Communication         |
| <b>OS</b>   | Operating System                    |
| <b>DAP</b>  | Debug Adapter Protocol              |
| <b>SSA</b>  | Static Single Assignment            |
| <b>CFG</b>  | Control Flow Graph                  |
| <b>DAG</b>  | Directed Acyclic Graph              |
| <b>3AC</b>  | Three Address Code                  |
| <b>DSL</b>  | Domain Specific Language            |
| <b>VM</b>   | Virtual Machine                     |
| <b>AST</b>  | Abstract Syntax Tree                |
| <b>RKO</b>  | Robot Karol Online                  |
| <b>DFG</b>  | Data Flow Graph                     |
| <b>DGIR</b> | Dungeon Intermediate Representation |

## Overview of Tools Used

- Github Copilot (<https://github.com/features/copilot>)
  - Thesis
    - \* Structural suggestions
    - \* Wording fixes
    - \* Literature research
    - \* Bibtex citation generation
    - \* Generation of initial version of pseudocode in Listing 56 and grammar in Listing 53
    - \* Consistent word styling, replacing `\texttt{}` with specific highlighting based on context.
  - Implementation
    - \* Code refactors
    - \* Documentation
    - \* Fixes and refinements for code in several areas including, safety checks for Serialization / Deserialization, cleanup of Reaching-Definitions, test implementations
    - \* Architectural input
- LanguageTool (<https://languagetool.org/>)
  - Thesis
    - \* Spell checking

## Abstract

Computer-science education has a long history of utilizing pedagogical programming environments to teach programming skills. Most programming environments target distinct levels of complexity, offering appropriate learning curves for pupils at different learning-levels. Early environments provided domain-specific textual languages designed to lower the barrier to entry, abstracting away most of the complexity associated with industrial programming languages. Later programming environments introduced visual programming languages, which result in greater student retention and motivation, quickly dominating the field of educational programming.

Modern educational programming environments extended on these concepts and introduced support for multiple programming languages in the same environment, with the goal of easing the transition from visual programming languages to textual ones. To facilitate the support for multiple languages, most environments compile to a simplified representation, which is then executed in an interpreter or virtual machine. Currently, all environments use their own program representation, requiring unique compiler implementations, execution models and validation logic to support their representation-semantics and format.

Meanwhile, industrial compiler infrastructure saw the introduction of flexible, domain-agnostic IRs such as MLIR and MimIR. These offer extension mechanism designed to allow developers to integrate their Domain Specific Language (DSL) easily, while making use of existing analysis and optimization pipelines. Nowadays, MLIR is used across a wide variety of compiler tools, providing a more streamlined development process in which application developers can focus on domain specific abstractions.

This paper presents *DGIR*, an extensible IR, designed for use in arbitrary educational environments, combining the simplicity of the educational IR Bril, with the flexibility of the extension system of the industrial IR MLIR. Through its highly compact and explicit API surface, developers can easily define domain-specific types and operations, shaping the IR to fit the semantic needs of the target language as direct as possible. Furthermore, Dungeon Intermediate Representation (DGIR) delivers an extensible and debuggable interpreter, allowing for easy integration into other applications. We show the effectiveness of DGIR as a simple and extensible IR through concrete usage examples, comparing it to existing industrial and educational IRs.

## 1 Introduction

With the widespread adoption of computer-based workflows and digitalization throughout schools and workplaces, digital competence is an increasingly important topic. Still, only 49% of the German population aged 14 and above meets the criteria for proficiency in basic digital competencies[11, p. 21]. The relevant areas, as defined by the D21 initiative, are:

1. Information and data competence
2. Communication and collaboration
3. Creation of digital media
4. Safety and wellbeing
5. Problem-solving competence

Simultaneously, 66% of the population feel that their own digital competence does not need to be improved, either because they are not interested or because they do not feel the need to[11, p. 22]. Even among the younger population (14-29 years old), almost half are deficient in one or more of these five key areas[11, p. 23]. This discrepancy between the need for digital competence and the willingness to acquire it is a problem that needs to be addressed in order to prepare the population for the digital future. Currently, only 30% of the population feel that schools teach them enough digital competence [11, p. 21], with 87% of active learners stating sources outside of school as their main way of acquiring digital competence [11, p. 22].

To address this, it is important to improve the way digital competence is taught in schools, starting earlier and building long-term motivation[4]. While programming is not directly applicable to all professions, it is a foundational skill that helps pupils understand the underlying principles of technology. It requires and teaches skills in all the relevant basic digital areas outlined by the D21 initiative, with almost all skills being applicable to other aspects of personal and professional life.

However, a significant pedagogical challenge remains: the transition from block-based programming environments (such as Scratch or Blockly), which lower the barrier to entry, to more complex text-based environments (such as Java). The transition from block-based tools to text-based programming requires bridging a conceptual gap that current pedagogical tools often fail to address. Even though hybrid approaches ease this transition [3], they often suffer from a lack of context and cohesion in their problem-solving activities.

The Probus project(Section 3) tries to bridge this gap by providing story-driven challenges instead of isolated ones, with the goal of increasing long-term engagement and learning success among pupils [21]. To achieve this

goal, Probus supplies a dungeon-crawler with multiple development frontends such as Blockly and Java, enabling the transition from block-based to text-based programming in a sequential progression. Initial problem-solving and algorithmic-thinking skills can be gained through block-based programming. Switching to a text-based environment then allows pupils to slowly build up their syntactical understanding of programming languages, by reimplementing and adapting their existing solutions. Pupils with basic prior knowledge can immediately target the more advanced textual frontend.

Because of its reliance on the Java platform, in its current form, the Probus project lacks the ability to extend its text-based training to more than one language. However, having the ability to teach and learn multiple programming languages through the Probus project would provide multiple benefits

- Modern technology stacks rely on multiple technological building blocks, oftentimes including different programming languages. Diversifying the learning experience should therefore provide a more solid foundation to build upon.
- While Java is a widespread language across modern curricula, based on pupil proficiency, languages with varying levels of syntactic complexity, such as Python or C, might be desirable to provide the optimal learning experience.
- Should the future decision be made to port the Probus project from Java to another programming language, building on a foundation that can handle programming languages independently of the implementation language would significantly facilitate such a transition.

These requirements place the project in a broader Intermediate Representation (IR) landscape that ranges from educational representations to larger domain-specific compiler infrastructures. Educational IRs such as Bril emphasize a compact and readable surface language, making them well suited for teaching, experimentation, and rapid prototyping[27]. At the other end of the spectrum, MLIR provides a highly extensible, hierarchical framework with rich support for domain-specific dialects and compiler tooling[19].

In this paper we present DGIR, an IR that is positioned between these design points. Its basic design aims to remain approachable for student developers while still supporting domain-specific operations and structured control flow. Rather than adopting the full complexity of industrial compiler infrastructure, DGIR keeps its API and representation intentionally compact, human-readable, and tooling-agnostic, so that it can be integrated into educational systems without imposing a steep learning curve.

At a high level, this thesis contributes the following:

- A rough overview of educational and domain-specific IRs, with Bril and MLIR serving as the central reference points.
- The design of DGIR as a lightweight IR that combines educational accessibility with selected ideas from more expressive compiler frameworks.
- An implementation-oriented demonstration of how DGIR reduces the complexity of existing technologies, while retaining the most relevant abstraction capabilities.

## 2 Overview of Educational Programming

This section establishes the historical and motivational foundation of educational programming environments and their evolution from simple text based environments to modern, multi-paradigm tools. It also introduces Robot Karol Online (RKO) as a design reference for the Probus project’s target featureset(Section 3).

### 2.1 History and Motivation

The history of computing is characterized by a continuous increase in the layers of abstraction between the user and the underlying hardware. While early computing required direct interaction with low-level commands and hardware-specific operations, the advent of Graphical User Interface (GUI) and higher-level programming languages has significantly reduced the technical barrier to entry. This progression toward higher levels of abstraction has facilitated the widespread adoption of technology but has simultaneously created a widening conceptual gap for learners transitioning between different levels of programming complexity.

Ecosystems and platforms, such as feature-rich operating systems, mobile computing devices (e.g., smartphones), and the Internet of Things (IoT), have further abstracted increasingly complex technology from end-users. With an increase in accessibility, technology became more widespread across all aspects of life, and technical skills a necessary qualification for large portions of the job market. To address these requirements, computer science education has become increasingly important in schools and educational institutions. It aims to teach students the fundamental principles behind modern technology and how to interact with it.

One aspect of computer science education is teaching programming skills. Programming is a fundamental skill that allows individuals to create software, applications, and websites, as well as automate tasks and solve problems using computers. Learning programming also helps students develop critical thinking, problem-solving, and logical reasoning skills. For this reason many educational institutions have started to include programming in their curricula, using various programming languages and tools to teach students the basics of coding. Over the years, many educational programming environments have been developed to make learning programming more engaging and accessible for students of all ages, with early examples including Logo[1] and Karel the Robot[24].

#### 2.1.1 LOGO

One of the earliest and most influential educational programming environments is Logo, developed in the late 1960s by Seymour Papert and his colleagues at Bolt, Beranek and Newman, Inc and the MIT AI Lab. It was a language designed for learning and exploration with a focus on mathematics, meant to teach a way of thinking about computers, learning and expressing what one is doing[30, p. 4]. Logo used a coding language that was easy to learn and understand, which allowed students to create simple programs with callable procedures and variables.

Listing 1 shows an example of a simple Logo program that generates random sentences by picking words from predefined lists. The keywords *TO* and *end* are used to define procedures, with *OUTPUT* returning a value from the procedure and *PRINT* displaying output to the user.

```
1 TO NOUN
2 OUTPUT PICK [BIRDS DOGS WORMS DONKEYS GEESE CATS [GUINEA PIGS]]
3 END
4
5 TO VERB
6 OUTPUT PICK [HATE TRIP BITE LOVE]
7 END
8
9 TO ADJECTIVE
10 OUTPUT PICK [RED PECULIAR JUMPING FAT FUZZY [FUZZY WUZZY]]
11 END
12
13 TO SENGEN
14 PRINT (SENTENCE ADJECTIVE NOUN VERB ADJECTIVE NOUN)
15 SENGEN
16 END
```

Listing 1: Simple LOGO program that prints random sentences[30, p. 5].

Invoking the procedure *SENGEN* would produce random sentences as seen in Listing 2.

Later on, Logo also supported turtle graphics, which allowed students to create visual programs by controlling a so-called "turtle" that could move around the screen and draw shapes. Logo was specifically designed to be easy to learn and use, with a simple syntax and a focus on exploration and creativity. It focused on interactivity

```

1 RED GUINEA PIGS TRIP FUZZY WUZZY DONKEYS
2 PECULIAR BIRDS HATE JUMPING DOGS
3 FAT WORMS HATE PECULIAR WORMS
4 FAT GEESE BITE JUMPING CATS

```

Listing 2: The output of the program Listing 1[30, p. 5].

and immediate feedback, allowing students to see the results of their programs in real-time, hiding tedious details such as memory management and complex syntax. This was uncommon at the time, as computers were quite limited in terms of processing power and memory[30, p. 18].

These design choices made Logo a popular choice for teaching programming in schools and educational institutions, and it has influenced many other educational programming environments that followed.

### 2.1.2 Karel the Robot

Karel the Robot is another early educational programming environment, developed by Richard E. Pattis in the early 1980s. It was designed to teach programming concepts using a simple robot that could move around a grid and perform basic actions such as picking up and putting down objects. Karel used a simple programming language that was easy to learn and understand, allowing students to create programs that controlled the robot's actions.[24] While it inherited some concepts from Logo, such as simple syntax and ease of use, it focused more on procedural programming and control structures, such as loops and conditionals and did not form a complete programming language like Logo did[5].

Later on, many variants of Karel the Robot were developed, including Karel-3D which was an extension of the original Karel idea into a 3D environment[24]. Another notable variant is Robot Karol, with its latest version 3.0 being released in 2015 by Ulli Freiberger as a German educational programming environment for schools[8]. Robot Karol was a translation and adaptation of the original Karel the Robot concept into German, with some modifications to better suit the needs of German schools.

The Listing 3 gives an example of a Robot Karol program that defines a procedure to place 3 bricks in a row, and then uses that procedure in a loop to fill a room with bricks until it hits a wall.

```

1 Anweisung Lege3Ziegel
2   wiederhole 3 mal
3     Hinlegen
4   endwiederhole
5 endeAnweisung
6
7 wiederhole solange NichtIstWand
8   Lege3Ziegel
9   Schritt
10 endwiederhole

```

Listing 3: Simple Robot Karol program[8].

### 2.1.3 Scratch

While Logo and Karel the Robot were influential in the early days of educational programming environments, they have largely been supplanted by more modern environments such as Scratch. Scratch was developed in the mid-2000s by the Lifelong Kindergarten Group at the MIT Media Lab[23], with the goal of making programming more accessible and engaging for children and beginners[22].

Instead of a text-based programming language, Scratch uses a visual programming language that allows users to create programs by dragging and dropping blocks that represent different programming concepts and actions. This approach makes it easier for beginners to understand programming concepts without the burden of syntax errors or other technical details. Scratch also includes a wide range of features and tools that allow users to create interactive stories, games, and animations, making it a popular choice for teaching programming in schools and educational institutions worldwide, with translations into over 70 languages[22].

In direct comparison to Karel, Scratch offers a more engaging and versatile platform for learning programming concepts, which fosters a more fruitful learning experience for students[26].

### 2.1.4 Blockly

Scratch popularized the concept of visual programming using blocks, leading to the development of other block-based programming environments such as Blockly. Blockly is a web-based visual programming environment developed by Google that allows users to create programs using a block-based interface[9]. It is designed to be easy to use and understand, making it a popular choice for teaching programming concepts to beginners and

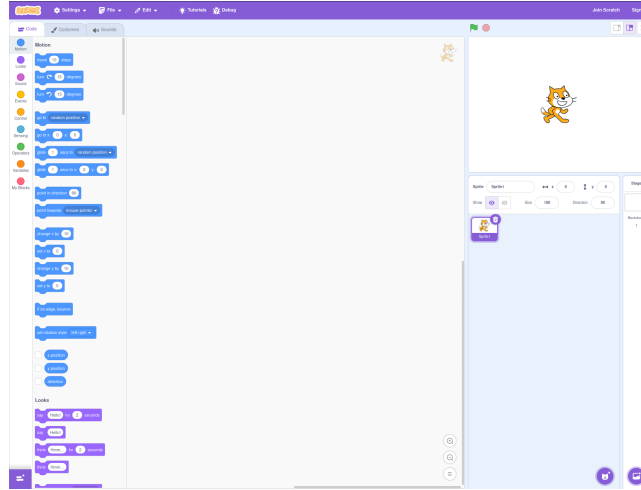


Figure 1: Scratch 3.0 Programming Interface

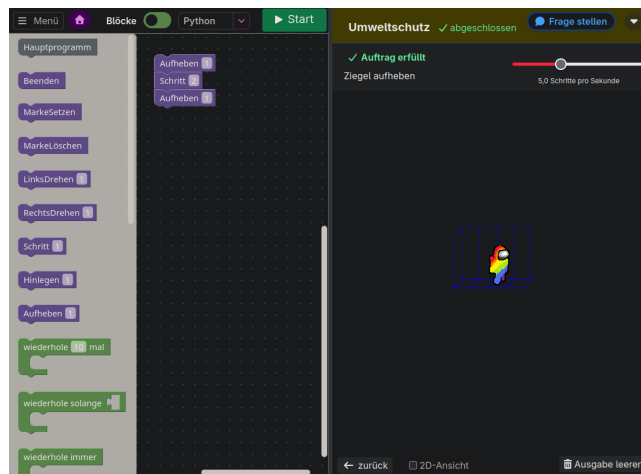


Figure 2: Robot Karol Online Programming Environment[18]

children. Blockly supports a wide range of programming concepts and actions, including variables, loops and conditionals, and can be used to generate code in multiple programming languages such as JavaScript, Python, and PHP[10].

It eliminates many of the complexities of text-based programs, such as syntax errors, through its drag-and-drop programming style, providing an intuitive environment and fostering creativity[25].

### 2.1.5 Robot Karol Online

RKO is an implementation of Robot Karol for the web by Quinn Li[18]. Development of RKO started in 2021, running fully in modern browsers. The goal was to evolve the existing Robot Karol implementation by Freiberger, providing a playful entry to the topics of algorithms, sequences, loops and conditional execution, targeting curricular use with children.

One of its most distinguishing features, and its primary evolution, is the support for multiple text-based programming languages—such as Python and Java—in addition to a Blockly environment. Not only does it allow for the compilation and execution of these languages, but it also supports automated conversion between them.



Figure 3: Robot Karol Online Blockly Code

The simple walk and pickup routine, which can be seen in Fig. 3, can be converted to its Robot Karol DSL equivalent through the press of a button (Listing 4). Java (Listing 5) and Python (Listing 6) translations are supported as well.

```
1 Aufheben
2 Schritt(2)
3 Aufheben
```

Listing 4: Equivalent Robot Karol DSL code to Fig. 3.

```
1 class Programm {
2     Robot karol = new Robot();
3
4     void main() {
5         karol.aufheben();
6         karol.schritt(2);
7         karol.aufheben();
8     }
9 }
```

Listing 5: Equivalent Java code to Fig. 3.

```
1 karol = Robot()
2
3 karol.aufheben()
4 karol.schritt(2)
5 karol.aufheben()
```

Listing 6: Equivalent Java code to Fig. 3.

Any changes implemented using one of the programming languages can be translated to every other supported programming language automatically. This improvement significantly decreases the conceptual gap between block-based and text-based programming languages, and lowers the barrier to entry when switching to another language.

## 2.2 Robot Karol Online: Technological Foundation

To realize its language switching behavior, each supported language comes with a script that can transpile a Robot Karol DSL Abstract Syntax Tree (AST) to its target language equivalent. During the compilation of every language other than Robot Karol DSL, two results are returned: the op-codes for the Virtual Machine (VM) and the Robot Karol DSL equivalent. This way, the latest valid form of any program is in memory when a language switch happens. It also means that most of the time three versions of the program exist: the selected language, the Robot Karol DSL and the compiled program in op-code form. The VM used to execute the compiled bytecode, follows a simple stack based design, with a small set of low-level and game specific operations (Listing 7). Its support for data types is limited to numbers, eliminating the need for any sort of typechecking.

This approach keeps the VM extremely compact and simple, lending itself perfectly to execution in a computationally limited environment like a browser. Compilation is handled without an intermediate representation, going from the ASTs directly to op-codes. Therefore, all compilers need to implement steps like destructuring control-flow (if-else statements to jumps and branches) separately, making the process more bugprone.

Comparing the goals of the Probus project(Section 3) to the capabilities of RKO, one notices a significant pedagogical and technological overlap. Both RKO and Probus aim to provide an environment that eases the transition from block-based to text-based programming, both support Java and both use a gameplay loop,

```

1 type Op =
2   | ActionOp // Movement, Brick and Marker interactions
3   | SenseOp  // World checks (e.g. wall in front)
4   | JumpOp   // Jump to target op-code
5   | BranchOp // Conditional branch to target op-code
6   | CallOp   // Jump to target op-code and open new stack frame with arguments
7   | ReturnOp // Return to caller and push return value onto previous stack
8   | OperationOp // Arithmetic operations
9   | ConstantOp // Push constant to the stack
10  | LoadOp // Push value of named variable onto op stack
11  | StoreOp // Store top value of stack into named variable
12  | CompareOp // Compare top values of stack
13  | AddRobotOp // Add robot to game world
14  | PopOp // Pop top value from stack
15

```

Listing 7: Op codes used by RKO’s bytecode.

centered around solving problems in sequential levels. Porting the VM, compilers and op-code structure from RKO to the Probus project would be comparatively straightforward, with the translation feature requiring a base-language covering all language features required by the Probus project.

However, this approach is not desirable for multiple reasons:

- **Abstraction:** RKO’s bytecode lacks the structural primitives necessary to represent complex control flow directly, resorting to unstructured control flow instead. This makes reasoning about compiled code much more difficult, as well as slowing down compiler development.
- **Domain-Specific:** While the opcodes used by RKO largely satisfy the requirements of the Probus project, implementing them as core components of the execution system prevents other projects from using the VM and bytecode.
- **Compiler Complexity:** Writing compilers for the IR should be as simple as possible. While programs compiled to the RKO bytecode format are checked for correctness during execution, all static analysis has to be performed by the compiler, increasing compiler complexity considerably.
- **Features:** While RKO’s bytecode offers sufficient features to model algorithms operating solely on unsigned integer values, adding support for additional numeric or structured data types would require substantial changes to the bytecode format and VM design.

Still, many of the design decisions made for RKO offer great insights into a potential workflow implementation. Section 4 will expand on the compiler concepts explored in the context of RKO and what role modern IRs play during the compilation of applications.

### 3 The Probus Project & Current Limitations

While block-based programming environments such as Scratch lower the barrier to entry, help with the transition to text-based languages, and promote motivation, creativity, and computational thinking, two challenges remain[21, p. 1]:

1. The transition from block-based to text-based environments
2. The isolated form of exercises, rather than coherent challenges, which enhances engagement and learning outcomes

The Probus project aims to address these challenges, while building on the insights on learning success and engagement, gathered by projects like Scratch. It incorporates a story-driven progression, embedded in a 2D dungeon crawler, reusing the same game world, characters, and core mechanics, providing structural continuity across workshop levels[21, p. 2].

It provides multiple programming environments, designed for its three target audiences

1. **Blockly-Dungeon:** Pupils aged 10-12 with no prior programming experience
2. **Java-Dungeon:** Pupils aged 11-14 with prior experience in block-based programming environment
3. **Advanced-Dungeon:** Pupils aged 15-16 with prior knowledge of text-based languages such as Java

As the name suggests, the *Blockly-Dungeon* is based on the Blockly programming language and introduces basic algorithmic thinking. The *Java-Dungeon* transitions to text-based programming and applies core Java constructs. The *Advanced-Dungeon* focuses on more complex programming concepts and open-ended problem-solving tasks[21, p. 2].

The goal is to make the transition from low-complexity concept to higher-complexity ones easier by enabling the pupils to progress through the environments and tasks sequentially, while keeping the same context.

#### 3.1 Limitations

The Dungeon architecture consists of two primary components: a game runtime and a frontend responsible for supplying code to the engine. This architecture enabled the usage of multiple programming environments such as the Blockly web frontend, or an extension for a code editor.

The current code pipeline looks roughly like this:

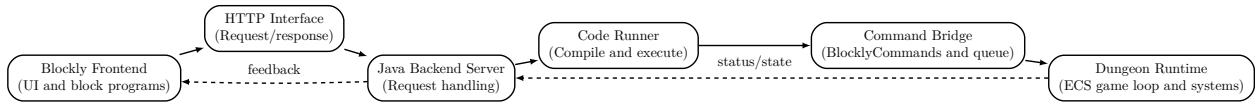


Figure 4: The execution-flow from frontend (in this case Blockly) to game.

While the *Java Backend Server* would be capable of receiving and handling any programming language, in practice the *Code Runner* limits the Dungeon to Java code only. The reason for this limitation lies in the way source code is compiled and executed. In its current form the compilation and execution pipeline process follows these steps

1. Format boilerplate Java code with the received user code
2. Compile source code using the system java compiler
3. Load the compiled class into the running instance
4. Call the *"execute"* method on a separate thread

During the execution, all commands are recorded into a work-queue, which then gets worked on in game time. This system has the advantage that it is simple to use, requires no custom compilers, and can execute user code at the speed of the game, since both run on the JVM. It also allows calling game classes from the user-code directly, without the need for indirection, allowing for code completion using the engine code as a reference.

However, this approach also comes with major disadvantages:

- Since we rely on loading Java classes during runtime, Java is the only supported language
- The game cannot be ported to another programming language without redesigning the execution model
- No support for debugging of the running code

- Code-completion needs to expose major parts of the engine source code to work correctly with existing code editor plugins
- Synchronization between the game and user-code thread is inconsistent, with some actions such as checks, needing to wait for previous code to finish running while others run fully asynchronously

Overcoming these limitations and adding support for more programming languages requires multiple fundamental changes to the code compilation, code execution and code-engine interactions. The main challenge is how to unify the code execution in a way that allows for portable, controlled and ideally debuggable execution, while keeping the game and user-code execution synchronized.

While it would be possible to compile each programming language with its corresponding compiler and create separate executables, this has several disadvantages

- **Complexity:** Synchronization between the different executables would need to be handled via a mechanism supported by all languages, such as shared memory, web requests or named pipes.
- **Portability:** The game would need to ship with compilers for every programming language on every deployed platform, increasing size. Furthermore, deploying to platforms without support for a given programming language would make them unusable.

On the other hand there are major advantages

- **Language Support:** As long as a compiler is available, any language, no matter how complicated, could be used, provided a library with the necessary Inter Process Communication (IPC) logic is provided.
- **Performance:** Running the programs in their native and optimized environments likely performs better than an in-game interpreter.
- **Workflow:** Using popular compilation and execution workflows also enables use of tools such as code editor plugins and debuggers.

Using IPC one could ensure that long-running operations block the user-code execution until they are finished and spawning or killing of threads would be no more complicated than calling the appropriate Operating System (OS) function.

Another approach would be to use a common textual IR or bytecode format, which is then executed using an interpreter in the game itself. In contrast to a bytecode format which is mostly meant for execution in a virtual machine and therefore as concrete and compact as possible, an IR is an algorithmic representation at a higher level of abstraction, which can be targeted as a common intermediate step for syntactically and semantically different languages.

This approach would come with several advantages:

- **Common Runtime:** None of the languages need to port the IPC logic and all behavior can be modeled through domain specific operations.
- **Control:** Execution and compilation is controlled by the game and can more easily be customized if needed.
- **Debugging:** Code debugging can be achieved in any form needed by the game and tooling surrounding it.
- **Portability:** Since the interpreter and compilers for the languages can be written in the same language as the game, portability is equally good as that of the game itself.

While these advantages work well to fix many of the shortcomings of the current implementation and make the game more portable than using the platform native compilers and debuggers, other challenges present themselves

- **Complexity:** While synchronization is in some ways simplified since we do not need to handle IPC, using a common representation also requires a custom compiler for every language. Termination of the interpretation is also more complex since cleanup and state management has to be handled manually.
- **Performance:** Matching the performance of modern runtimes or native code execution is almost impossible in the context of a project of this scale. Therefore, execution of complex programs will most likely be significantly slower than before.
- **Workflow:** Using a custom compiler and execution model means that most of the existing tooling for the languages will not be usable or only in limited form. Using standard debugging protocols like the Debug Adapter Protocol (DAP) makes the process easier but also requires conforming closer to standard practices.

These challenges would make this approach a lot less attractive for many projects, but are mostly non-issues in the context of the Probus project. Since the Probus project is intended as a research project for computer science students, implementations and improvements of components like the compilers, common representation, runtime and tooling, can be used as teaching opportunities. Therefore, having the opportunity to extend and implement this rather complex systems is as much an opportunity as it is an initial hurdle.

Performance is also a non-problem since the user-code developed to solve the tasks is comparatively simple and mostly does not exceed a couple dozen lines. Workflow problems remain as the biggest hurdle with this approach.

### 3.2 Target

In addition to its use in the Probus project, the target IR will potentially be used in an educational context, independent of the Dungeon, and should therefore not model the functionality of any domain directly, but serve as a foundation which can be extended upon. In this context portability is of the essence, since students should be free to write compilers in any programming language while targeting the same IR.

Based on these requirements and the need to support arbitrary programming languages and computations, with domain-specific operations like controlling a game character, the IR must meet the following requirements:

- **Simplicity:** Generating IR output should be as simple as possible. Syntax which would require a custom byte or textual format is therefore not an option.
- **Extendability:** All operations modeled by the IR should follow a canonical form, independent of the semantic meaning. Operations are modeled based on a shared set of capabilities, while validity checks and semantic meaning are implemented separately. For example, all control flow should be decomposed into a standard set of primitive properties.
- **Abstraction:** In order to reduce the implementation complexity for the compilers targeting the IR, structural properties like scopes, functions and structured control flow should be as simple to represent in the IR as possible. The goal is not to represent the computations in a way that is close to the hardware.
- **Readability:** The IR should be in a human-readable form that is easy to understand during inspection and, potentially, teaching.
- **Portability:** Parsing the IR should ideally be supported by most languages out of the box.
- **Correctness:** Programs in IR form must be in a verifiable and structurally sound form. As long as a program is structurally sound, it must be verifiable and if it passes validation should represent a semantically correct computation. A typesafe syntax is preferred for this reason as well, with all values carrying type information, requiring the use of correctly typed data in all situations.
- **Debuggability:** IR programs should be mappable to their source code, a strict requirement for features like breakpoint debugging.

These requirements also clearly show why this paper will focus on an IR instead of a bytecode format. A potential bytecode format simply would not provide the level of readability and portability required by the project it will be used in.

Before continuing with the architectural evolution and design decisions made for DGIR, the next chapter will establish a contextual baseline concerning compiler infrastructure and intermediate representations, required to understand DGIR's final approach and referenced work.

## 4 Compiler Infrastructure and Intermediate Representations

This section discusses basic compiler infrastructure, IR forms and how IRs are used in modern compilers. Section 4.7 and Section 4.8 present the main references used during the design DGIR and their basic applicability to the goals of the Produs project.

IRs are data structures commonly constructed during compilation of a program, designed to make optimizations easier[31, p. 1]. Furthermore, IRs not only serve optimization, but also enable different language-specific frontends and target-specific back-ends to share a common IR. This enables compilers to leverage existing infrastructure and therefore significantly reduce the effort of implementation[31, p. 2].

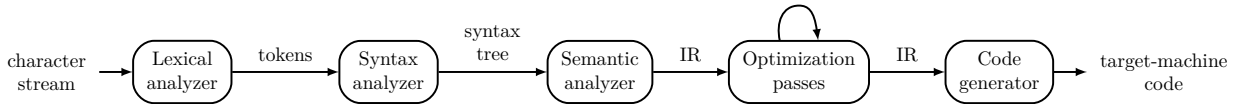


Figure 5: "A typical compiler structure split into phases." [31, p. 2]

Figure Fig. 5 shows a typical compiler structure with its individual stages and the data structures passed between them. While the analysis of the source syntax and semantics are mostly performed in a domain-specific representation, later steps can be performed using the IR.

Probably the most well known example of such a shared compiler infrastructure is the LLVM[15] project. It is a compiler framework providing a common, low-level IR called LLVM IR, with the capabilities to generate code for many platforms. Today, many of the most commonly used compilers are based on, or use components of the LLVM project. These include:

- **Clang**[14]: A front-end for C, C++ and Objective-C
- **Rust**[13]: A modern programming language commonly using LLVM for code generation
- **Kotlin**[12]: A multiplatform language based on the JVM, but with a LLVM backend for compilation to native binaries

While these compilers target LLVM IR eventually, most of them implement their own IR during previous compilation steps, like "MIR" in the case of Rust, which only lowers to LLVM IR for the final code generation.

### 4.1 Linear and Graphical IRs

When talking about IRs, it is important to note that there are many different forms an IR may appear in. The following taxonomy was taken from Stanier and Watson (2013):

*The taxonomy consists of three categories in which IRs can exhibit characteristics.*

*The first category is structure. The structure of an IR can be divided into two broad subcategories:*

- *Linear. The IR represents pseudocode for a machine. This varies from relatively high-level instructions to low-level instructions, similar to assembly language.*
- *Graphical. The IR represents program information in the form of a graph.*

*We choose to split graphical IRs into two further subcategories: cyclic and acyclic, based on these graph theoretic properties.*

*The second category is the dependence information represented:*

- *None. The IR is not designed to highlight any dependence information.*
- *Control. The IR represents relationships explicitly in terms of the control dependences between variables or sequences of instructions in the program.*
- *Data. The IR represents relationships explicitly in terms of the data dependences between variables or sequences of instructions in the program.*
- *Hybrid. The IR highlights both control and data dependence information.*

*The third category is the program content contained within the IR:*

- *Full. The compiler is able to generate target code with only the information present in the IR.*
- *Partial. The compiler requires more information, stored externally from the IR, in order to generate target code.*

| IR                    | Structure           | Dependence | Content |
|-----------------------|---------------------|------------|---------|
| Polish Notation       | Linear              | None       | Partial |
| Extended Polish       | Linear              | None       | Full    |
| Triples/Quadruples    | Linear              | None       | Partial |
| 3AC                   | Linear              | None       | Full    |
| SSA                   | Linear              | Data       | Partial |
| GSA                   | Linear              | Data       | Full    |
| AST/Parse Tree        | Graphical (acyclic) | None       | Full    |
| DAG                   | Graphical (acyclic) | Data       | Full    |
| CFG                   | Graphical (cyclic)  | Control    | Full    |
| Superblocks           | Graphical (cyclic)  | Control    | Full    |
| SSA Graph             | Graphical (cyclic)  | Data       | Partial |
| DFG                   | Graphical (cyclic)  | Data       | Partial |
| PDG                   | Graphical (cyclic)  | Hybrid     | Full    |
| PDW                   | Graphical (cyclic)  | Hybrid     | Full    |
| VDG                   | Graphical (cyclic)  | Data       | Full    |
| VSDG                  | Graphical (acyclic) | Data       | Full    |
| Click's IR            | Graphical (cyclic)  | Hybrid     | Full    |
| Dependence Flow Graph | Graphical (cyclic)  | Data       | Full    |

Table 1: "Classification of IRs According to the Taxonomy [...]"[31, p. 5]

Table 1 also provide their classification of the most popular kinds of IRs given the previous taxonomy.

While table Table 1 may suggest that each of the different IRs exist as separate concepts from another, most of the time more than one IR is used alongside each other or sequentially. A modern example of such a combination of IRs is *MLIR*[19] which provides a linear IR based on the Static Single Assignment (SSA) form, performing analysis on the Control Flow Graph (CFG) and Directed Acyclic Graph (DAG) which results from the SSA dominance and structural rules defined by the IR[16].

Section 4.2, Section 4.3 establish the basics of IR notations, with subsequent chapters exploring more advanced IR concepts required to understand modern IRs.

## 4.2 Polish Notation

Polish notation is one of the earliest forms of linear IR developed for early stack-based machine architectures, allowing for efficient representation of mathematical expressions.

$$E = (5 + 3) \times (10 - 2)$$

The infix expression from above, translated to the prefix polish notation looks like this

$$5\ 3\ +\ 10\ 2\ -\ \times$$

while the postfix notation looks like this

$$\times\ +\ 5\ 3\ -\ 10\ 2$$

The notation can simply be scanned from left to right, placing the operands on the stack and applying the last operator to the operands on the stack immediately, once there are sufficient operands on the stack[31, p. 5].

RKO bytecode is based on an evolved variant of Polish Notation called *extended Polish* that added non-arithmetic operations such as branching, loops and assignment. To realize operations like function calls with arguments, first the arguments are placed on the stack and then the function call is performed. Since the arguments could be pushed onto the stack in any order, it is important that both callee and caller agree on the same data layout. While modern compilers use register based instructions instead of stack based ones, when the number of available registers is less than required to pass all arguments, arguments are stored on the stack to substitute the missing registers. The exact behavior is defined by the compiler and called *calling convention*.

## 4.3 Three-Address Code

Three Address Code (3AC) is a linear IR which consists of a list of instructions with at most three operands per instruction, meaning that built-up arithmetic expressions like  $5 + 3 \div 10$  are not allowed[2]. Instead, such

an expression must be broken up into two separate instructions

$$\begin{aligned} t_0 &= 3 \div 10 \\ t_1 &= t_0 + 5 \end{aligned}$$

LLVM IR uses a form of 3AC which looks closer to assembly and contains type information but expresses the same basic meaning.

```
%t_0 = sdiv i32 3, 10
%t_1 = add i32 %t_0, 5
```

#### 4.4 Static Single Assignment

SSA describes a variant of IRs, meaning it does not describe a language in itself but a technique that can be applied on an IR[31, p. 7]. Its rules enforce that each variable is only defined (assigned) once, making data-dependency information easily discoverable, since each variable points to its exact point of definition. When control flow branches, where each branch assigned a different value to a variable  $V$  accessed after control flow merges, a so called  $\phi$ -function is inserted. Based on the control flow path taken, the function will create the final definition of  $V$ .

Fig. 6 illustrates how this is handled in practice: when a program branches, the branches define distinct values which are then accessed by the  $\phi$ -function based on which control flow edge leads into the merged block.

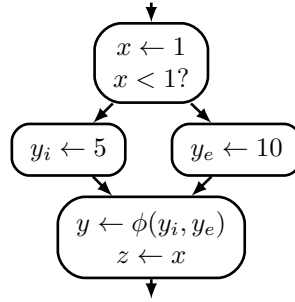


Figure 6: SSA CFG with value propagation guided by dominance rules and  $\phi$ -function. Note:  $x$  is defined in the first basic block and visible in the last without explicit propagation.

The SSA form makes dependency management and many optimizations so much simpler, that it is widely used across most modern compilers. However, executing  $\phi$ -functions directly is not possible, as they miss the concrete condition required to select the correct value. Therefore, further lowering is required before code generation can be performed. This conversion is called gating[31, p. 8].

One intuitive use-case for  $\phi$ -functions is the induction variable present in for loops. These variables need to be updated for every repetition of the loop and since redefinition (reassignment) is forbidden by the rules of SSA, a different mechanism is required to update the value. This is where  $\phi$ -functions come into play, as they allow us to redefine the induction variable with a new value every time the loop body is executed. An example of this can be seen in Listing 8 where an infinite loop is modeled using LLVM IR. When entering the loop for the first time through the basic block *LoopHeader*:(not shown for simplicity), the  $\phi$ -function will select 0 as the initial value of the induction variable, or the value of *nextindvar* when control flow is passed from the loop body(*Loop*:). Since the incremented *nextindvar* is passed back to *indvar* by branching to the beginning of the basic block, a new value will be visible to the loop.

```
1 Loop:           ; Infinite loop that counts from 0 on up...
2 %indvar = phi i32 [ 0, %LoopHeader ], [ %nextindvar, %Loop ]
3 %nextindvar = add i32 %indvar, 1
4 br label %Loop
```

Listing 8: Infinite counting loop in LLVM IR.

Listing 9 demonstrates a possible implementation of the infinite loop, violating the SSA rules by redefining the induction variable during loop execution. This approach is closer in appearance to high-level languages like C and therefore more intuitive to most developers, but loses the strong guarantees provided by using the SSA form.

```

1 LoopHeader:
2   %indvar = 1
3 Loop:      ; Infinite loop that counts from 0 on up...
4   %indvar = add i32 %indvar, 1 ; error: variable redefinition is not allowed
5   br label %Loop

```

Listing 9: Non-SSA implementation of Listing 8

While redefining value is not allowed using the SSA form, variables will likely be reassigned during execution. The actual register layout and value lifetime is only realized later, once the virtual registers (prefixed with %) are converted to real registers. At this point a compiler very well might decide to reuse the same register for all uses of `%indvar` and `%nextindvar`, as long as it does not change the semantics of the program.

MLIR[19] is another IR which uses the SSA form, but handles its pseudo-assignments using structural properties of its basic blocks, as can be seen in Listing 10. Here, redefinition of values is handled by declaring control flow dependent values as arguments supplied when entering a basic block (Line 6 Listing 10). When branching to another basic block within a region (which is required for any transition from one block to another, even when executing a linear list of basic blocks), the branching operation has to supply the basic block arguments as part of the invocation (line 6 & line 14 Listing 10).

```

1 ^bb1
2   %c0 = arith.constant 0 : i32 // Initial value of induction variable ( LLVM IR ==> [ 0, %LoopHeader] )
3   %c1 = arith.constant 1 : i32 // Step size ( LLVM IR ==> Numeric literal in add instruction )
4   cf.br %loop_body(%c0) : (i32) -> () // Start the loop by branching to the loop body with the initial value 0
5
6   ^loop_body(%indvar: i32): // Loop basic block with induction variable as parameter
7     %nextindvar = arith.addi %indvar, %c1 : i32
8     cf.br %loop_body(%nextindvar) : (i32) -> () // Branch carries the new induction variable

```

Listing 10: Same loop as Listing 8, translated to MLIR.

Using this approach avoids the conceptually more complicated use of  $\phi$ -functions, makes data-flow easier to read and avoids the gating problem by making value selection part of the explicit control flow.

Another approach to this problem can be seen in the educational intermediate representation Bril[27], where, similar to MLIR, the pre-branch operations explicitly set the values used during execution of the target basic block[28]. In contrast to MLIR, the usage of SSA form for Bril programs is not a strict requirement and therefore does not embed its arguments passing into the basic block and branching signature. This allows Bril to support SSA as an optional feature, while leaving the regular non-SSA syntax untouched.

```

1   zero: int = const 0;
2   one: int = const 1;
3   set indvar zero;
4   .loop:
5     indvar: int = get;
6     nextindvar: int = add indvar one;
7     set indvar nextindvar;
8     br .loop;

```

Listing 11: Same loop as Listing 8, translated to textual Bril in SSA form.

SSA is a strong tool used across almost all modern compilers, but it is only one aspect of a fully featured analysis pipeline. Linear IRs using SSA form are mostly stored directly in CFGs or CFG-like structures, used to derive further information like domination relationships. See Section 4.5 for an explanation about what CFGs are and how they are used for analysis.

## 4.5 Control Flow Graph

CFGs are directed graphs consisting of nodes containing single instructions or a *basic block*. Basic blocks are a list of instructions with a single *entry* and *exit*. Edges represent possible control flow transitions and every CFG must contain an *entry* and an *exit* node[31, p. 11]. Fig. 6 shows the control graph for a simple branching program in SSA form. Note how the nodes of the CFG contain the linear IR operations making up the program code itself. Listing 12 gives the program in its fully linear form, as seen before construction of the CFG.

An important property of CFGs is that control flow can only enter at the first instruction of a *basic block* and exit at the last. Branches therefore may only occur at the end of the block. The IR modeled in Listing 12 does not follow the exit rule explicitly. The *entry* block ends with two branching operations: the first one is the explicit *fjump* in line 4, the other is the implicit fall-through from *entry* to *if* in case the *fjump* is not performed. However, in this case, the violation can easily be fixed when constructing the CFG by adding a *fall-through* edge from the *fjump* operation to the *if* block. If the IR allows placement of control flow operations anywhere

```

1 .entry:
2   x ← 1
3   cond ← x < 1
4   fjump cond .else    // Jump to .else label if the condition is false
5 .if:
6   yi ← 5
7   goto .merge
8 .else:
9   ye ← 10
10  goto .merge
11 .merge:
12  y ← φ(yi, ye)
13  z ← x

```

Listing 12: Linear representation of CFG in Fig. 6

in its labeled regions, regions would have to be split into separate basic blocks at the control flow operations (Listing 13)

Further, CFGs contain all information about the dominance relationship between basic blocks. In a CFG, a basic block (node)  $X$  is said to dominate a basic block  $Y$ , if  $X$  is visible on every path from *entry* to  $Y$ . If  $X$  is distinct from  $Y$  we say that  $X$  strictly dominates  $Y$  [7]. Subsequently, all values defined in the dominator  $X$  are also visible in  $Y$ . This property allows the use of values across basic blocks without explicit propagation, as can be seen in Fig. 6 where the value  $x$  is used to define  $z$  outside its defining block.

|                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1 .entry: 2   x ← 1 3   cond ← x &lt; 1 4   fjump cond .else 5   // Invalid in basic block: branch must be last 6   altcond ← x &lt; 2 7   fjump altcond .else 8 .if: 9   y<sub>i</sub> ← 5 10  goto .merge 11 .else: 12  y<sub>e</sub> ← 10 13  goto .merge 14 .merge: 15  y ← φ(y<sub>i</sub>, y<sub>e</sub>) 16  z ← x </pre> | <pre> 1 .entry: 2   x ← 1 3   cond ← x &lt; 1 4   fjump cond .else 5 .entry_1: // Split into two valid basic blocks 6   altcond ← x &lt; 2 7   fjump cond .else 8 .if: 9   y<sub>i</sub> ← 5 10  goto .merge 11 .else: 12  y<sub>e</sub> ← 10 13  goto .merge 14 .merge: 15  y ← φ(y<sub>i</sub>, y<sub>e</sub>) 16  z ← x </pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Listing 13: Splitting region before constructing CFG so that exit rule holds.

CFGs are mostly constructed from linear lists of operations, with construction following straightforward transformation rules. To simplify construction and maintenance, IRs like LLVM IR and MLIR treat their code blocks like basic blocks. This allows for the construction of CFG-derivatives like dominator graphs, without transforming the IR into an explicit CFG graph first, since all the information is already present in the linear representation.

## 4.6 Data Flow Graphs and Def-Use Chains

Data Flow Graph (DFG) are directed graphs where nodes represent operations and edges the flow of data from the result of the operation to the input of another. In contrast to the CFG, a DFG does not contain information about the ordering of operations and therefore cannot be used to generate target code directly. Using the CFG and DFG alongside each other opens the door to more effective optimizations [31, p. 12-13].

Building on this concept, a def-use chain represents the data flow from variable definitions to their usages. Def-use chains are built atop linear IRs. Their purpose is to connect the results of operations to all their uses, or inversely uses to their definition (use-def chain), allowing for fast traversal during optimizations and rewrites. Use-def chains are explicitly added to linear IRs in SSA form, building a so called *SSA graph* [31, p.13].

MLIR stores its explicit use-def chains by handling all operand-slots and values as nodes in a linked list. Every time a value is assigned as an input to an operation, the corresponding operand-slot is added to the use-list of that value. And since all values only have one definition, they lead directly to the operation that defined them, while retaining the ability to move the value to a different operation during rewrites, without having to update the use-def chain. MLIR and LLVM IR maintain numerous bidirectional graphical relationships between their data structures, making traversal of the IR substantially more efficient.

IRs like the graphical SSA IR *Sea Of Nodes* (or *Click's IR* named after its original author) encodes both the data flow and control flow information in the same IR [6], therefore not needing to manage the def-use chain explicitly.

Building on these concepts, we will now discuss practical examples of modern IRs that serve as the foundation of DGIR.

## 4.7 Low-Level, Flat Representations

Low-level, flat representations like LLVM IR are a widespread form of IRs used across a wide range of industrial compilers. Having a low-level flat IR allows the representation of computations as close to the hardware as possible, without needing to introduce target specific code. This enables the reuse of optimization strategies across all platforms, before generating native target code. Being flat means that the IR cannot represent hierarchical structures directly, with LLVM IR supporting no nested constructs like structured control flow or local functions. Abstractions like functions are the highest level of abstraction available. Because programs are linear lists of operations, interpreters can be designed with a numeric program counter, realizing branches and functions calls by jumping to a specific instruction index.

Another low-level, flat IR that is drastically simpler than LLVM IR, is Bril. Bril, the Big Red Intermediate Language, is an intermediate representation designed for the *Advanced Compilers*  $\lambda$  course at the Cornell University[28]. Programs written in Bril are linear, typed, minimal and based on the JSON format, making them tooling agnostic. Its design goals follow a simple rational: students should learn with a hands-on structure and have the ability to start quickly without having to learn a large Application Programming Interface (API) associated with industrial compiler frameworks. While it does not intend to serve as a general purpose IR that could be used for professional compiler engineering, it is "pretty good" for the use case of teaching compiler classes[29].

Bril programs consists of five syntactic elements, each optionally containing position fields to map it to a source position. Listing 14 demonstrates a simple Bril program, showcasing all syntactic elements.

- **Program:** The top level program containing all functions.
- **Type:** Primitive types like *int* and *bool* and parametric types like pointers wrapping another type.
- **Function:** A first order procedure containing a list of instructions. It has a name and optionally a list of arguments and return type.
- **Label:** Marks a named target position for control transfers in an instruction sequence.
- **Instruction:** A unit of computational work. These contain the opcode name and optional arguments required to execute the operation, including the name of the destination variable for the result, the type of the result, the names of the argument variables, the names of the referenced functions, the name of the referenced labels and a value literal for a constant.

Any program must follow this structure strictly to be syntactically well formed[28, Syntax Reference]. Semantically well formed programs have to follow common rules of statically typed languages (static type for variables, assigned values must be of same type, etc.) and used variables must be defined on all control flow paths leading to their usage[28, Well Fromedness].

```
1 {  
2   "functions": [{  
3     "name": "main",  
4     "args": [],  
5     "instrs": [  
6       { "op": "const", "type": "int", "dest": "v0", "value": 1 },  
7       { "op": "const", "type": "int", "dest": "v1", "value": 2 },  
8       { "op": "add", "type": "int", "dest": "v2", "args": ["v0", "v1"] },  
9       { "op": "print", "args": ["v2"] }  
10    ]  
11  }]  
12 }
```

Listing 14: Bril program adding 1 and 2, before printing the result[29].

Bril's core language consists of a minimal set of types(64 bit integer and boolean) and operations required to represent basic computations and control flow. Listing 15 is written purely using the core language features of Bril and shows a simple factorial program using Bril's textual format. It reads very similar to the equivalent LLVM IR code while being significantly less verbose. In comparison to LLVM IR, which is syntactically and semantically similar, Bril requires substantially less type annotations. This has the advantage of increasing readability by reducing visual noise.

One important distinction between LLVM IR and Bril is that Bril does not use SSA form by default. This means that previously defined values like induction variables can be reassigned during loop execution simplifying the syntax further and allowing for an easier onboarding experience. Bril features multiple extensions, adding

functionality like raw memory manipulations, floating point operations, dynamic typing and SSA semantics. Using the SSA extension adds the *set* and *get* instructions explored in Section 4.4 Listing 11, allowing programs to be represented in SSA form.

|                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1 @main(input: int) { 2   res: int = call @fact input; 3   print res; 4 } 5 6 @fact(n: int): int { 7   one: int = const 1; 8   cond: bool = le n one; 9   br cond .then .else; 10 .then: 11   ret one; 12 .else: 13   decr: int = sub n one; 14   rec: int = call @fact decr; 15   prod: int = mul n rec; 16   ret prod; 17 } </pre> | <pre> 1 @fstr = private constant [4 x i8] c"%d\0A\00" 2 declare i32 @printf(i8*, ...) 3 4 define i32 @main(i32 %input) { 5   %res = call i32 @fact(i32 %input) 6   %0 = call i32 (i8*, ...) @printf(i8* @fstr, %res) 7   ret i32 0 8 } 9 10 define i32 @fact(i32 %n) { 11   %cond = icmp sle i32 %n, 1 12   br i1 %cond, label %then, label %else 13 then: 14   ret i32 1 15 else: 16   %decr = sub i32 %n, 1 17   %rec = call i32 @fact(i32 %decr) 18   %prod = mul i32 %n, %rec 19   ret i32 %prod 20 } </pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Listing 15: A simple factorial program written in Bril, give in its textual representation(left)[29] and its LLVM IR equivalent(right).

While Bril does fulfil the simplicity and portability constraints set for DGIR, it relies heavily on fixed syntactic building blocks like functions and parametric types with a single type parameter, without the possibility to extend or replace them with domain-specific equivalents. This severely limits its use for projects targeting a different or higher level of abstraction. For example, writing a frontend for a language supporting higher-order functions requires immediate lowering to first-order functions, potentially introducing the need for an additional IR to support a more gradual lowering process. While current Bril extensions add further operations and types, structural extensions (e.g. nested functions) would not fit its current design, as there are no general rules as to how structural primitives have to be built up. Further, Bril does not supply a builtin extension mechanism, meaning that all extensions have to be implemented through modifications of the core source code.

For DGIR, Bril’s decision to use JSON as its IR format is the most relevant design decision, as it fulfills the requirement of portability and simplicity, being easy to parse and read, with wide support for JSON handling across all major programming languages. However, Bril’s limited flexibility when it comes to structured control flow and missing extension mechanisms, limits its adaption by DGIR.

## 4.8 Multi-Level, Hierarchical Representations

While low-level IRs like LLVM IR have proven their usefulness as the last step during compilation before interpretation or emission of target code, lowering to low-level IRs directly, makes source-level analysis very difficult. This is a problem since many language-specific optimizations and checks are easier to perform at a higher-abstraction level. One case where a higher level of abstraction is advantageous would be superfluous matrix operations like double transpositions. A double transposition does not have an effect on the matrix and therefore can be omitted completely from the program. Identifying this optimization opportunity in a fully lowered and inlined program is significantly harder compared to high-level abstractions.

Furthermore, lowering complex programs to low-level IRs directly poses a significant challenge, therefore prompting the use of higher-level IRs as an intermediate step in many compilers.

To address these shortcomings and provide a common framework for optimizations and lowering strategies, MLIR was developed as part of the LLVM project[16], introducing a higher-level syntax than LLVM IR and adding the concept of regions, which can be attached to operations. Regions consist of a list of basic blocks, enabling the definition of operations representing concepts like structured control flow, scopes, and functions in a familiar hierarchical style. MLIR choses to provide no types or operations as part of the IR specification, instead relying on extensions to provide definitions for all types and operations, including functions and and program scopes(see Listing 17). It provides an extension mechanism called dialects, allowing for the definition of operations, types, attributes, lowering passes and rewrite strategies. Further, all operations can provide custom verification, extending the builtin analysis with domain-specific semantics. The idea behind MLIR is to provide compilers the ability to model DSL features directly, initially compiling to a high-level representation, before progressively stripping away abstractions, running type inference and optimizing code. Once all code is lowered

to the dialects that are part of the MLIR project, existing lowering and optimization passes can be used, until finally everything is lowered to LLVM IR (Listing 17, Listing 18).

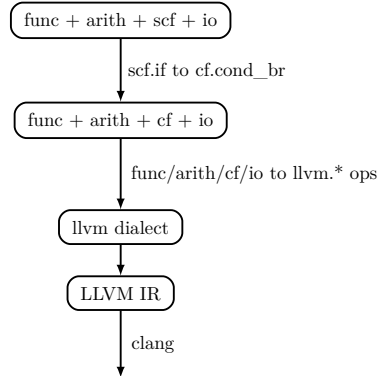


Figure 7: Lowering steps to go from Listing 17 to Listing 18

Operations in MLIR can represent arbitrary semantics, fully opaque to other dialects or operations. To enable this, all operations must adhere to a basic set of semantic and structural limitations, which are enforced by representing all operations using the same class. This means that dialect specific operations do not define custom data types for their operations through inheritance, but instead define semantic wrappers around the underlying operation class, with custom serialization, validation and typing. This has the advantage that users can supply custom operations with operation-specific validation, custom assembly formats, and an appropriate API, while MLIR can guarantee the structural verifiability of the program.

In addition to operations and types, users can define rewrite strategies using pattern matching to handle domain-specific optimization opportunities explicitly, or to lower operations from a high-level dialect to a lower-level one. MLIR code can be lowered to any other dialect without the requirement to target machine code, which is advantageous when the target-code is itself a high-level language like HLSL.

```

1 "op.name"(%operand1, ...) [successor1, ...] ({region1}, ...) <{attr1 = val1, ...}> : (type1, ...) -> result_type
  ↪ loc(<sourceLocation>)

```

Listing 16: Generic MLIR operation syntax.

The generic syntax in Listing 16 contains all the information that can be contained in any given operation, be it user defined or built-in. Operations are made up of:

- **Value-Operands:** The SSA values used as arguments for the operation.
- **Block-Operands:** The successor blocks of an operation. Only the last operation in a basic block can have successors as dictated by the CFG structure of regions.
- **Regions:** Regions are MLIR’s way of structuring control flow similar to *scopes* in languages like *C*. They contain a list of basic blocks forming an enclosed CFG, meaning that branching operations may not reference basic blocks from other regions, including regions attached to the same operation. Information about when and if regions are entered during execution is entirely determined by the semantics of the owning operation and not stored in the textual IR.
- **Attributes:** The attribute dictionary holding static values. These are used when declaring constants where the actual value is not a SSA value but a static constant. Attributes are *uniqued* during construction, ensuring attributes with equal storage use the same storage. Attribute objects serve as a thin wrapper around the underlying storage object.
- **Types:** The types of the value-operands and result of the operation. Types are *uniqued* during construction, ensuring all types with equal parameters share the same storage and are comparable via pointer equality.
- **Source-Location:** The source location maps an IR operation to its original source file and location therein.

As pointed to earlier, MLIR enforces strict limitations guiding how operations can and cannot be used through the IR, mainly extending the control flow rules dictated by CFGs to handle the hierarchical structures modeled through regions. While MLIR allows domination (Section 4.5) to extend to nested regions, making values from containing basic blocks visible to their child regions, branching between basic blocks of different regions is strictly forbidden. This ensures that the CFG contained in each region behaves as expected. However,

```

1 module {
2   func.func @main(%input: i32) {
3     %res = func.call @fact(%input) : (i32) -> i32
4     io.print(%res) : (i32) -> () // Fictional io dialect
5     return
6   }
7
8   func.func @fact(%n: i32) -> i32 {
9     %one = arith.constant 1 : i32
10    %cond = arith.cmpi sle, %n, %one : i32
11
12    %result = scf.if %cond -> (i32) {
13      scf.yield %one : i32
14    } else {
15      %decr = arith.subi %n, %one : i32
16      %rec = func.call @fact(%decr) : (i32) -> i32
17      %prod = arith.muli %n, %rec : i32
18      scf.yield %prod : i32
19    }
20
21    return %result : i32
22  }
23 }

```

Listing 17: High level MLIR code for Listing 15

```

1 module {
2   llvm.func @printf(!llvm.ptr, ...) -> i32
3   llvm.mlir.global private constant @fmt("%d\0A\00") :
4     ↪ !llvm.array<4 x i8>
5
6   llvm.func @main(%input: i32) -> i32 {
7     %res = llvm.call @fact(%input) : (i32) -> i32
8
9     %fmt_ptr = llvm.mlir.addressof @fmt : !llvm.ptr
10    %zero = llvm.mlir.constant(0 : i64) : i64
11    %str = llvm.getelementptr %fmt_ptr[%zero, %zero] :
12      ↪ (!llvm.ptr, i64, i64) -> !llvm.ptr, !llvm.array<4 x
13      ↪ i8>
14    llvm.call @printf(%str, %res) vararg(!llvm.func<i32
15      ↪ (!llvm.ptr, ...) > : (!llvm.ptr, i32) -> i32
16    %ret = llvm.mlir.constant(0 : i32) : i32
17    llvm.return %ret : i32
18  }
19
20  llvm.func @fact(%n: i32) -> i32 {
21    %one = llvm.mlir.constant(1 : i32) : i32
22    %cond = llvm.icmp "sle" %n, %one : i32
23    llvm.cond_br %cond, ^then, ^else
24    ^then:
25      llvm.return %one : i32
26    ^else:
27      %decr = llvm.sub %n, %one : i32
28      %rec = llvm.call @fact(%decr) : (i32) -> i32
29      %prod = llvm.mul %n, %rec : i32
30      llvm.return %prod : i32
31    }
32  }
33 }

```

Listing 18: Listing 17 lowered to LLVM dialect before translating to Listing 15

in the case of operations like functions, visibility of values from outer scopes might not be desirable, for which an *IsolatedFromAbove* trait is available, preventing values from being passed into regions of an isolated operation. The hierarchical structure of regions and domain-specific operations allows computation to be represented at a much higher level of abstraction compared to Bril(?), making them a desirable addition to DGIR’s tooling and language.

Apart from regular language compilers, currently, MLIR is used across a wide variety of software, including areas like machine learning, hardware analysis and quantum computing. MLIR’s modularity, highly extendable nature, and high-level abstraction make it a valued tool for applications aiming to represent domain-specific problems with the ability to optimize for different target architectures.

Apart from MLIR, very few IRs provide an infrastructure as flexible and extendable as needed by modern DSL compilers. One recent approach standing out in this regard is *MimIR*[17], a Sea Of Nodes based IR using a form of typed  $\lambda$ -calculus, providing a general base language with an expressive type system, supporting high-order polymorphism, dependent types and a type-safe extension mechanism. While MimIR improves on many of the shortcomings of MLIR, especially in regard to its limited type-system, its syntax and extension mechanism require a more advanced understanding of formal computer science, making its approach less viable for an introduction to IRs. Therefore, MimIR will not be discussed further in the context of this work.

The design of DGIR is largely based on the previous work done by Bril and MLIR, as they are some of the very few IRs providing all the features required for Produs’ design goals. The following chapter will discuss the architectural evolution and design decisions made for DGIR and how its design is based on the IRs discussed in this section.

## 5 DGIR Methodology and Architectural Evolution

This chapter defines the methodological frame used to design and evaluate DGIR and then presents the architectural decisions that emerged from that process.

### 5.1 Methodological Framework

The central research question of this thesis is:

How can the simplicity of Bril be combined with the flexible and hierarchical design of MLIR to create a practical educational IR with a lower API complexity?

To answer this question, the design and evaluation of DGIR are guided by four primary objectives:

- **Simplicity:** Reduce conceptual and implementation complexity compared to MLIR.
- **Flexibility:** Support domain-specific extension through dialect-like mechanisms.
- **Robustness:** Enforce structural correctness through explicit validation and constrained representations.
- **Performance neutrality as a design stance:** Prioritize ease of use over low-level implementation tricks.

Success is defined qualitatively by the ability to adopt core MLIR design principles (dialects, regions, explicit operation semantics, typed attributes) while omitting high-complexity mechanisms that are unnecessary for the intended educational and prototyping context.

### 5.2 Evaluation Design

This thesis uses a qualitative, artifact-centered evaluation. The goal is not to optimize runtime performance, but to evaluate whether the resulting IR and API are simpler while retaining the required expressive power. For the comparison against existing compiler infrastructure, three baselines are used:

- Bril with its current extension ecosystem and TypeScript implementation.
- MLIR with its table-gen and C++ workflow for defining and using custom dialects.
- LLVM with its current implementation of LLVM-IR.

Evidence for its claims of improved simplicity, flexibility, and applicability in real compiler usage is drawn from the implementation and use of DGIR in real development settings:

- development of a Java compiler frontend that targets DGIR, and
- development of an IR interpreter operating on DGIR.

These artifacts are treated as integration cases that expose usability, extensibility, and robustness constraints.

The comparison against Bril and MLIR is organized along the following dimensions:

- **API complexity:** amount of required boilerplate, number of required concepts, and explicitness of required extension metadata.
- **IR simplicity:** regularity and readability of serialized IR, and predictability of object models.
- **Extensibility:** effort needed to add new types, attributes, and operations.
- **Robustness:** enforcement of valid construction paths and explicit verification behavior.

While the comparison gives qualitative answers to the research question, evaluation of the results is subjective and centers on the idea that DGIR’s API should enforce correct usage while reducing complexity as much as possible. Performance or minimal code size are not central concerns. Therefore, no quantitative user study and no benchmark-based runtime performance evaluation are included. Raw implementation size metrics (e.g., lines of code) are not used as primary evidence, because they are context-sensitive and can be misleading when interpreted in isolation.

### 5.3 Threats to Validity

Due to this thesis’ lack of quantitative evaluation and focus on integration driven evaluation, several threats to the validity of our claims remain:

### 5.3.1 Construct validity.

The key constructs (simplicity, flexibility, robustness) are primarily assessed through implementation experience and structural analysis rather than controlled measurement. This introduces interpretation risk. To mitigate this, claims are tied to concrete architectural choices and observable API behavior.

### 5.3.2 Internal validity.

The same implementation process both produces and evaluates the artifact, which can introduce author bias. Mitigation is provided by direct baseline comparison against two established systems and by grounding claims in concrete integration tasks.

### 5.3.3 External validity.

Findings are based on an educational/prototyping context and two concrete integration artifacts. Generalization to production-grade compiler infrastructures should therefore be interpreted cautiously.

## 5.4 Architectural Design Evolution

The following sections document the final architectural decisions and the reasoning that led to them. The emphasis is on design choices that directly support the methodological goals defined above.

### 5.4.1 Extensions

MLIR's extension mechanism, *dialects*, provides the core capability needed by DGIR. MLIR dialects provide a grouping mechanism for operations, types and attributes under a common namespace. During dialect registration, the details of all operations, types and attributes are added to the current *MLIRContext*, allowing them to interact with MLIR systems such as the printing and parsing, type checking and validity checks transparently. Dialects can be loaded at runtime and rely on type erased representations of the concrete implementation, meaning that dialects can be compiled separately from MLIR making them portable. DGIR therefore adopts dialects as first-class extension units for user-defined operations, types, and attributes, but with a reduced feature surface. This reduction in feature surface mainly focuses on MLIR's capability to handle opaque operations, types and attributes; a feature not required for DGIR's intended use-cases.

Before discussing individual components, one core syntactic concept is required: the identifier (*ident*). Identifiers provide globally unique names for DGIR entities, while allowing contextual overlap across categories (for example, an operation and a type may share the same textual name if their usage sites are distinct). The syntax is shown in Listing 19.

```
1 <ident>      ::= <name> ('.' <name>)*  
2 <name>      ::= <letter> (<letter> | <digit> | '_' )*  
3 <letter>    ::= 'a' | ... | 'z' | 'A' | ... | 'Z'  
4 <digit>     ::= '0' | ... | '9'
```

Listing 19: DGIR identifier syntax.

Types and operations in the builtin dialect may omit the namespace prefix to reduce verbosity in serialized programs.

MLIR's dialects register all their elements once they are loaded by an *MLIRContext*, making it the responsibility of the user to add all elements to the dialect during construction. While this approach works as intended, requiring the user to explicitly call the *addOperations/Types/Attributes* helpers leaves the exact point of registration up to the user, creating uncertainty as to the correct time of registration. To mitigate this, DGIR's dialects require the user to implement abstract methods returning the individual lists of elements, which are then used by a centralized *register* method. The registration can then be triggered at any point during execution, making it possible to obtain dialect instances without registering their elements. However, if needed, operations can also be registered separately from their target dialect by changing the registry directly.

Since MLIR is built around the usage of rewrite passes, dialects provide multiple mechanisms to extend their functionality through so-called *interfaces*. These allow MLIR to reuse rewrite strategies on all elements providing an implementation of the interface. Through this mechanism MLIR provides runtime polymorphism for dialects, but also for operations, types, and attributes. This means that interfaces can be attached to existing elements and dialects without owning them, making it possible to extend their semantics to support additional features. This mechanism was omitted by DGIR to further reduce its complexity, since it is not the goal to

support IR-transformations as a first-class feature. However, adding runtime polymorphism to DGIR would be possible and could be realized similarly to MLIR.

MLIR requires dialects and their elements to be registered through a specific `MLIRContext`, which serves as the global registry for all elements. This allows MLIR to handle multiple different sessions at once, each with its own set of loaded dialects. Handling such complex setups is not a goal of DGIR, allowing us to turn the DGIR-dialect-registry(`DGIRContext`) into a singleton that can be accessed from anywhere and therefore does not require explicit handling throughout the user-facing API.

### 5.4.2 Values

One of DGIR’s most fundamental design decisions was how to model values consumed and produced by operations. LLVM IR represents constants and dynamic values through a common value hierarchy, optimizing for uniformity of use, where every constant literal, result of a previous instruction, or basic block is treated as a value. Although this maximizes uniformity, it also blends entities with different semantic roles, adding complexity during analysis.

Following MLIR and Bril, DGIR separates dynamic values from constant data, avoiding the ambiguous semantics of value objects in LLVM IR by guaranteeing that a value is always the result of an operation. Similarly to MLIR, constants are represented through attributes, while values remain tied to operation results or region arguments. This produces a clearer semantic model and simplifies reasoning about definition and use. Bril accomplished the separation of dynamic values and constants by defining two types of instructions, one with dynamic value inputs and a constant operation taking a value literal. This model limits the usage of constant data in Bril to specific operations, limiting their flexibility, while MLIR’s attributes can be attached to any operation, mixing dynamic and constant inputs as required.

A second central decision concerns SSA. DGIR intentionally does not enforce SSA form. This choice prioritizes readability and ease of use during development over aggressive optimization support, consistent with the project objectives. The resulting value model follows Bril-style redefinition with type consistency constraints at each definition site. This has the major advantage that values like collector variables do not need to be handled using the SSA compliant approach MLIR uses for its for-loops(Listing 20).

```

1 %sum = scf.for %iv = %lb to %ub step %step // final result %sum yielded after last iteration
2   iter_args(%sum_iter = %sum_0) -> (f32) { // iter_args binds initial values to the loop's region arguments.
3     %t = load %buffer[%iv] : memref<1024xf32>
4     %cond = arith.cmpf "ugt", %t, %c0 : f32
5     %sum_next = scf.if %cond -> (f32) {
6       %new_sum = arith.addf %sum_iter, %t : f32
7       scf.yield %new_sum : f32
8     } else {
9       scf.yield %sum_iter : f32
10    }
11    scf.yield %sum_next : f32 // can't write result to %sum_iter directly, need to yield all iter_args
12 }

```

Listing 20: MLIR loop example illustrating SSA accumulator handling via iter args.

Not enforcing SSA removes strict single-definition def-use chains. To retain practical rewrite support, DGIR stores use-lists for IR objects referenced by operations (for example values and blocks) and updates them eagerly when operands are added or removed.

Values carry explicit type information and are treated as unique handles within a program instance, ensuring that a unique value is always be represented by a single object. This supports comparison based on pointer-equality, as well as serialization and deserialization using unique value names.

### 5.4.3 Types

In contrast to Bril and LLVM IR, DGIR’s type system must remain extensible while supporting arbitrary type structures. A fixed closed set of kinds like the ones used in Bril and LLVM IR is too restrictive for this goal. Bril uses a fixed type union to represent all types, while LLVM IR uses inheritance, marking the specific type of the type-instance using an enum. Extending the type system of either requires direct modification of the type-classes, making this approach unfeasible for DGIR.

MLIR’s design separates abstract type metadata from instance-specific storage and relies on uniqueness storage indirection. This means that MLIR’s type objects consist of multiple different objects:

- The abstract type containing all the static metadata common to all instances of a type
- The type storage containing the type specific storage and a reference to the abstract type. This storage is unique and no two storage instances with the same content exists in the same `MLIRContext`.

- The type containing a reference to the type storage. This class serves as a semantic wrapper around the unique type storage.

This approach offers strong equality and memory-sharing properties, but introduces substantial implementation complexity, especially in regard to indirection of the memory model. For DGIR, the same conceptual separation is retained, but the user-facing type model is simplified.

Another source of indirection, which might cause confusion when using MLIR is the way parsing and printing is implemented for individual types. Instead of handling type-, attribute- and operation-parsing consistently inside the specific classes, type and attribute parsing and printing is handled by a central method defined by each dialect. While this is a valid approach, operations define their printers and parsers individually creating a conceptual gap between how types and operations are handled during serialization. Mapping of operations to their printers and parsers is handled through the type erased representation of an op, which holds a reference to the op's implementations. Grouping all operation specific code, including the parsing and printing in the op implementation, offers a clearer modular structure not seen with types. DGIR utilizes a more consistent style, where the serialization of types and operations is handled through the same mechanism.

The resulting requirements were:

- direct inheritance for type classes,
- separation of abstract type metadata from instance data,
- type-local serialization/deserialization logic,
- parameter-sensitive equality semantics.

While the implementation of an inheritance-based type system without the use of separate storage objects was straightforward, the retrieval process of abstract type metadata underwent multiple iterations. Abstract type information will be called `TypeDetails` from now on, the name it was given in DGIR.

**5.4.3.1 Abstract Type Data - Initial Approach** Because the current DGIR implementation is written in Java, MLIR's template-heavy C++ patterns could not be adopted directly, requiring a rethinking of the mechanism used to retrieve the abstract type information of DGIR's types. In MLIR, access of static fields and methods through template-code is used extensively during registration and to discover capabilities of types. During implementation of a new type, one is required to provide the static members accessed by these functions, which have to follow the exact syntax expected by the template-code. Accessing static members like this is not possible from generic Java code as it would violate Java's type-safety rules, which is not the case for C++. While the same behavior could be achieved through the use of reflection, doing so would sidestep all compile-time language guarantees in addition to making navigation between declaration and usage impossible.

These limitations boil down to the following criteria:

- The implementation of members providing metadata should be explicitly required by subclasses of *Type*, and therefore
- Instance methods must provide the metadata.
- Reflection should be mostly avoided.

An early approach used instance-provided descriptor factories, where the default constructed instance of a type had to provide an instance of a type-descriptor. This descriptor was then stored inside the actual abstract type of the type, providing the required runtime type information. Although functional, this introduced unnecessary indirection and made metadata flow harder to reason about. Constructing a default instance before registration of the type also meant that type instances could exist without a valid abstract type requiring additional checks throughout the code. Another disadvantage of the factory approach was that whenever the type storage needed to be accessed to create specific metadata for specialized types, the current instance had to be passed to the descriptor object, increasing bloat.

Therefore, the factory approach was dropped and replaced with fine-grained methods providing all metadata directly. During registration, a new type instance would be passed in, the metadata of which would then be accessed to populate the abstract type directly. While this reduced the indirection and increased ergonomics of the type API, using instances of types to populate the details referenced by the type instance, meant that types had to remain mutable so that they could be updated with their details after registration. This stands in contrast to the semantics of type objects, which should always represent the same information and was therefore overhauled in subsequent iterations. At this point DGIR still supported the concept of registered and unregistered types which was an artifact of MLIR's type system and its support for opaque types. A feature outside the scope of DGIR.

**5.4.3.2 Descriptor Objects** To separate type description from type instances cleanly, explicit type descriptor objects were introduced. Descriptors allow registration before first instantiation and enable strict checks for unregistered type usage. At this point DGIR dropped support for unregistered or opaque types in the TypeDetails registry completely.

Listing 21 shows the current details record. The key fields are the textual identifier and concrete Java class mapping used during deserialization. The `parameterizedIdFactory` reconstructs a concrete type from its parametric textual representation and is provided by the author through the type descriptor. Separating the type descriptors from the types themselves, solved the problems caused by Javas inability to access static members from generic code, while ensuring that type instances can be immutable and complete. For this reason, support for custom escaped expressions and integers were added to the supported type parameters.

```

1 record TypeDetails(
2     String ident,
3     String namespace,
4     Class<? extends Type> type,
5     Dialect dialect,
6     Function<Object, Boolean> validator,
7     Function<Pair<String, TypeDetails>, Type> parameterizedIdFactory) {}

```

Listing 21: TypeDetails record from current source code.

**5.4.3.3 Type Syntax** MLIR type printing and parsing is fully customizable, with the only limitation being that types must start with a `!` followed by the type identifier. This allows types to represent their data with any custom syntax making the textual type representations highly expressive. To achieve a similar level of customizability, the current type-syntax of DGIR follows familiar parametric notation with optional custom payloads for specialized forms, as shown in Listing 22 and Fig. 8. Initially types could only represent simple identifiers with basic parameter lists, but with the introduction of more advanced parametric types such as function-types, it became obvious that a more complex type syntax was needed to represent more complex semantics.

```

1 <type> ::= <ident> <type_params>?
2 <type_params> ::= '<' <type_param> ( ',' <type_param> )* '>'
3 <type_param> ::= <type> | <string_literal> | <number>
4 <string_literal> ::= '"' <any_char_except_quote>* '"'
5 <number> ::= '-'? <digit>+

```

Listing 22: Type syntax for DGIR types.

*func.func < "()" → ()>*

Figure 8: Parametric function type with custom syntax.

## 5.4.4 Operations

DGIR operations follow an MLIR-inspired structural model to support region-based control flow and uniform parsing/printing. While the operation and `op`(Paragraph 5.4.4.2) API did evolve over time to be more streamlined and concise, the general concept and design remained largely the same. Adopting MLIR’s structural model imposes limitations on what operation can represent, driving the operation architecture in a very specific direction. While operations can represent a wide variety of semantic concepts, how these concepts can be represented has to be constrained in order to guarantee structurally correct programs. Compared to MLIR, mutability and representational flexibility are intentionally reduced to lower conceptual overhead.

MLIR operations follow a strict set of constraints put in place to provide generalized verification of the IR structure and syntax, which are not needed by Bril, as it lacks the concept of regions and does not enforce the use of basic blocks. These limitations allow MLIR to extend the CFG based bodies of functions in LLVM IR, with the concept of regions, providing a more expressive syntax for high level abstraction. While Bril’s operation architecture is appropriate for use in a flat-linear IR, for DGIR the more flexible approach of MLIR’s operations is required. Adopting it will enable DGIR to represent arbitrary semantic concepts and computations at a high level. Furthermore, MLIR established multiple statically enforceable validity criteria, which should also be applicable to operations of DGIR, even though they might have to be adapted to fit the non-SSA value semantics.

Adopting a unified operation object representing all operations regardless of semantics also enables the reuse of the same printing and parsing logic for all operations. While this reduces the expressivity of serialized

operations compared to MLIR's custom assembly formats, serializing all operations using the same syntax fits Bril's design goal of being "*ruthlessly regular*" much more closely.

Another notable simplification is that each DGIR operation produces at most one result value. A decision that would substantially limit MLIR's operation abstractions, but does not significantly limit DGIR as we are not bound to SSA form and therefore can express complex dataflow more directly. These constraints significantly reduce API surface, with the trade-off of limiting flexibility.

```

1 final class Operation {
2     /** The type erased representation of the op, this operation is contained in. */
3     private final OperationDetails details;
4     /** The input values of this operation. */
5     private final List<ValueOperand> operands;
6     /** The input blocks of this operation (branch successors). */
7     private final List<BlockOperand> blockOperands;
8     /** The output of this operation. */
9     private final OperationResult output;
10    /** The attributes of this operation. */
11    private final Map<String, NamedAttribute> attributes;
12    /** The dynamic attributes of this operation. */
13    private final Map<String, NamedAttribute> dynamicAttributes;
14    /** The regions of this operation. */
15    private final List<Region> regions;
16    /** The block containing this operation. */
17    private Block parent = null;
18    /** The source location of this operation. */
19    private final Location location;
20 }

```

Listing 23: Contents of the Operation class in DGIR.

**5.4.4.1 Operation Details** As with types(Section 5.4.3), operations reference a details object that stores type-erased semantic accessors for validation and wrapper construction. Operations themselves do not contain any semantic info about the dialect-specific operation they represent. Accessing the semantic operation wrapper provided by the dialect implementation is therefore done via the hooks provided to the operation details during registration, removing the need to know the specific op-type at compile time. Since all operations are represented by the same underlying class, type checking for operations has to be performed through the indirection provided by the operation details, where either the **ident** or **type** of the containing op can be compared. During deserialization of operations, **idents** are mapped to their corresponding operation details, providing the **opFactory** required to construct the semantic wrapper for the dialect specific op from the raw operation.

For the creation of operation details from semantic wrappers, an initial reflection-heavy registration approach was replaced by explicit abstract methods and factory hooks. This made extension contracts more transparent and reduced hidden control flow.

```

1 record OperationDetails(
2     String ident,
3     Class<? extends Op> type,
4     Dialect dialect,
5     Function<Operation, Boolean> verifier,
6     Set<Class<? extends IOpTrait>> traits,
7     Map<Class<? extends IOpTrait>, Method> traitVerifiers,
8     Function<Operation, Op> opFactory,
9     Supplier<List<NamedAttribute>> defaultAttributes) {}

```

Listing 24: OperationDetails record from current source code.

**5.4.4.2 Semantic Wrapper** Concrete dialect semantics are represented through operation wrappers called **Op**. An op does not duplicate operation payload data; instead, it provides typed semantic access, validation logic, and operation-specific helpers around a backing **Operation** object. Consequently, an op (Listing 25) is a minimal base class, providing a description for an operation, similar to how type descriptors provide the semantic information about types(Section 5.4.3).

While initial implementations provided explicit serialization and deserialization hooks for both JSON and textual formats, these were quickly replaced by the generalized serialization approach described above. Extending ops to support custom serialization would have been possible, but was actively decided against in favor of reduced API surface and implementation complexity.

This design exposes required extension hooks explicitly through inheritance, providing compile time feedback on the correct implementation of an op. Not relying on static members improves API and dataflow transparency, as methods do not have to be accessed through reflection to provide generalized registration support. However, some optional metadata of ops (for example trait implementations) is still queried through reflection to reduce

```

1 abstract class Op {
2     public abstract Class<? extends Dialect> getDialect();
3     public abstract String getIdent();
4     public abstract Supplier<List<NamedAttribute>> defaultAttributes();
5     public abstract Function<Operation, Boolean> getVerifier();
6     public abstract Function<Operation, Op> getOpFactory();
7 }

```

Listing 25: Contents of the Op class in DGIR.

dialect boilerplate. In case ops should be made fully immutable later down the line, implementing a descriptor based registration mechanism similar to that of types and attributes is a viable option.

#### 5.4.5 Attributes

DGIR adopts MLIR’s attribute concept to represent literal or annotation data separately from dynamic values. This separation preserves a consistent definition/use model for values while still allowing operations to carry semantic parameters.

Attributes are represented as named entries in an attribute map and can contain arbitrary storage payloads. For example *IntegerAttr* from DGIR’s builtin dialect contain a single integer value and are additionally typed, making them *TypedAttributes*. This way operations without value inputs can produce values, which in turn have a clear definition site. Initially, all attributes were treated as typed attributes which was later revised to better represent arbitrary annotations needed by operations like *BinaryOp* to specify the concrete binary operation that should be performed. Furthermore, early revisions of DGIR stored all attributes in mutable maps. This stood in contrast to the semantics of operations like *BinaryOp*, which rely on the presence of specific attributes to be in a valid state. Therefore, later design revisions introduced a split between:

- Immutable, operation-defined attributes (allocated from static defaults), and
- Mutable dynamic attributes for optional annotations.

The immutable portion improves robustness by ensuring required semantic attributes are always present, while the dynamic portion preserves flexibility for workflows that require transient metadata during multi-stage compilation. MLIR based language frontends like the ClangIR(CIR)[20] need to attach AST references to their operations, as the AST carries necessary information for some transformation steps taken during the lowering process. However, the AST references are not inherent properties of the underlying operations and instead added and removed transitively. The dynamic portion was provided to serve such use-cases.

As with types and operations, attributes are identified by unique textual identifiers and are resolved through descriptor-backed registry mappings during serialization and deserialization. Similarly to types, attributes provide their own serialization and deserialization logic as they can represent arbitrary storage configurations. However, instead of providing the printing and parsing hooks through the attribute details, JSON serializers and deserializers are provided through annotations or in complex cases separate classes. While serialization could be handled uniformly for all attributes through generalized storage object semantics, this approach was not taken as it drastically increases the verbosity of attribute classes, and decreases readability of the IR.

#### 5.4.6 Operation Traits

DGIR provides a mechanism to describe common semantic properties and validation of operations via composition called *OpTraits*. They are based on the equally named concept from MLIR and fundamental to generalized operation semantics. An op trait describes a structural property of an operation and provides the means to verify that operations are used correctly in regard to this property.

These are some fundamental op traits MLIR provides and relies on during validation:

- **IsTerminator** This trait marks an operation as a terminating operation of a block. Validation of this trait checks whether this operation is the last in its block. During operation verification MLIR ensures that every block’s last operation has a **IsTerminator** trait.
- **IsIsolatedFromAbove** This trait marks an operation as isolated from above, signaling that values from parent regions are not visible to blocks in the contained regions.

Being able to compose operations from existing op traits does not only provide improved semantic validation, but also allows generalized validation- and analysis-steps to be implemented, improving the infrastructure’s reusability. Accessing traits of operations is done through their operation details (Listing 24) which stores all trait classes implemented by an op and the corresponding verification method of that trait. While MLIR relies

on compile time template-based discovery of all the implemented traits, DGIR uses reflection instead as there is no compile-time equivalent of variadic templates in Java, which would be required to port this feature directly. Instead, reflection is used to query all implemented interfaces extending from the common `IOpTrait` interface.

```

1 public interface IHasResult extends IOpTrait {
2     static boolean verify(Operation operation) {
3         if (operation.getOutput().isEmpty()) {
4             operation.emitError("Operation must have a result.");
5             return false;
6         }
7         if (operation.getOutputValue().isEmpty()) {
8             operation.emitError("Operation must have a result value.");
9             return false;
10        }
11        return true;
12    }
13
14    default Value getResult() {
15        return getOperation()
16            .getOutputValue()
17            .orElseThrow(
18                () ->
19                    new RuntimeException(
20                        "Expected operation to have a result value: " + getOperation());
21    }
22
23    default Type getResultType() {
24        return getResult().getType();
25    }
26 }

```

Listing 26: Implementation of the `IHasResult` trait promising the existence of a result value.

The op trait base interface only declares a `getOperation` method, providing methods defined in op traits with a convenient accessor to the operation implementing the trait. MLIR relies on multiple-inheritance to provide ops with the methods declared in op traits, meaning that every op trait implemented by an op will be one of its super classes. Java however does not allow multiple inheritance and therefore forces the implementation to use interfaces instead. Older Java versions would therefore prevent us from implementing the methods provided by the op trait in a reusable way as interfaces cannot provide function definitions, only declarations. Java 8 introduced default methods for interfaces allowing authors to provide an implementation for methods declared in an interface. In combination with the `getOperation` accessor, op traits can now provide the expected shared semantics without requiring the implementing op to provide any further method definition.

Listing 26 gives an example of a simple trait, providing semantic information about the operation result. Here we can see that similarly to MLIR, OpTraits provide additional verification through a static method called `verify`. Initially this function was an instance method with one argument: an object of the op trait type. The argument was used to provide distinct method signatures for each trait verification method, otherwise inheriting multiple traits would have caused redefinition issues. However, since reflection was used to access the verifier regardless of using static or instance methods, the implementation was changed to static verification methods instead. While it would be possible to provide a lookup mechanism for type trait verification methods, similar to the operation and type details, doing so would make the op trait API significantly less ergonomic. Therefore, reflection remained the mechanism of choice for op trait verification discovery.

Currently, op traits are the only mechanism providing generalized, reusable operation semantics and the usage of op traits has to be fully defined at compile-time. In case the need arises, for future iterations of DGIR, the addition a runtime polymorphism system, like the one provided by MLIR’s interface system, could be considered. This feature could however significantly increase the API surface of DGIR, while its uses in the intended educational contexts would most likely be minimal, and was therefore not considered relevant for the current goals of DGIR.

## 6 Results

This chapter presents the finished DGIR artifact and reports observed outcomes against the methodological dimensions defined in Section 5.2. All findings are qualitative and are grounded in implementation artifacts and integration experience from the Java compiler frontend and the DGIR interpreter. Interpretation of project success, limitations, and broader implications is deferred to Section 7.

The reported results are organized in two layers: the artifact representation detailing what was built and what it supports, and the claim-aligned outcome reporting focusing on what was observed relative to each methodological dimensions. The following artifacts provide the evidence base:

- The DGIR core implementation(Section 6.1.1).
- The Java compiler frontend targeting DGIR.
- The DGIR interpreter implementation.

### 6.1 Final DGIR Artifact

This section presents the final form of DGIR as implemented in this thesis. It focuses on the concrete outcome: IR form, extension model, and supported workflows.

#### 6.1.1 JSON IR Form

The only fully supported format for serialized DGIR programs is JSON, where every program must consists of a limited set of structural objects/elements, following a strict syntax. In the following sections we will discuss the JSON syntax of all DGIR elements. Every JSON syntax listings will be written using the following syntactic rules:

- The types of values will be written as plain text(e.g. String).
- If a field is optional, the field name will be appended with a ?.
- Values serialized as objects will be wrapped inside {}.
- With fields that can hold multiple possible types of value, each option is separated by | and all are enclosed in ().
- Arrays and objects are written using the default JSON syntax.

**6.1.1.1 Operations** **Operations** form the fundamental building block of all DGIR programs, with all possible operations using the same data-structure. DGIR does not allow for the definition of custom serialization behavior through **Op** definitions to keep the IR as consistent as possible. An operation may represent any semantic concept, including a unit of work, a structural primitive or a storage declaration. The semantic meaning of an operation is not encoded as part of the serialized IR and instead defined by the backing dialect implementation. Therefore, every operation must contain the ident of the defining op, so that the correct semantic wrapper can be constructed from the serialized representation. Listing 27 gives the final syntax for serialized operations.

```
1 {
2   "ident" : String,
3   "loc"? : String,
4   "operands"? : [ String, ... ],
5   "successors"? : [ String, ... ],
6   "attributes"? : [ { NamedAttribute }, ... ],
7   "dynamicAttributes"? : [ { NamedAttribute }, ... ],
8   "output"? : ( { Value } | String ),
9   "regions"? : [ { Region }, ... ]
10 }
```

Listing 27: DGIR JSON operation syntax.

```
1 {
2   "ident": "func.func",
3   "loc" : "Main.java:4:1",
4   "attributes": [
5     {
6       "name": "symbol_name",
7       "attr_ident": "stringAttr",
8       "attr_type": "string",
9       "attr_value": "main"
10    },
11    {
12       "name": "type",
13       "attr_ident": "typeAttr",
14       "attr_type": "func.func<\"() -> ()\">"
15    }
16  ],
17   "regions": [{ ... }]
18 }
```

Listing 28: Example function operation serialized to JSON

Most of the fields of an operation are optional and do not have to be included when they are not needed, reducing file-size and improving readability. Operations like the function op seen in Listing 28, can omit most of the fields present in the generalized syntax and only include what is essential. The `output` field can either be a string or a value output depending on whether the referenced value was defined by a previous value or is defined the first time. This first definition in the IR does not mark the first semantic definition of the value, but marks the first occurrence from top to bottom. Every operation has an optional source-location field(`loc`) which maps the given operation to its original source file. DGIR's DAP server relies on this information to provide step debugging functionality.

When targeting the interpreter's DAP server, the special source-location `<ignore>:-1:-1` informs the interpreter that the given operation should be skipped during step- or breakpoint-debugging.

**6.1.1.2 Attributes** `Attributes` serve as storage for operations that require arbitrary static data to model their semantics and are stored in dictionaries of `NamedAttributes`, mapping a unique identifier to a concrete instance. Since DGIR values must have a well-defined definition site before being used as arguments, operations like constants, that produce values from literals, have to store the concrete literal value as an attribute. This ensures that value semantics remain consistent and easy to reason about.

For operations, we differentiate between static and dynamic attributes, with the first serving as structural elements of an operation that have to be present in every instance of an op, while the second provide transitive storage, unique to every operation.

As part of the function definition(Listing 28), two static attributes are included; the `symbol_name` and the `type` (function-type) of the operation. The data of the attribute stored in the serialized named attribute, is inlined into the body of the named attribute, to avoid unnecessary nesting in the IR. The `Attribute` and `NamedAttribute` syntax are as follows:

```
1 {
2   "ident": String,
3   "type"?: String,
4   "..."?: Any
5 }
```

Listing 29: DGIR JSON Attribute syntax.

```
1 {
2   "name": String,
3   "attr_ident": String,
4   "att_type"?: String,
5   "att..."?: Any
6 }
```

Listing 30: DGIR JSON NamedAttribute syntax.

All fields of the inlined attribute in Listing 30 have to be prefixed with `"attr_"` to avoid naming conflicts between the fields of the two objects. During deserialization, all prefixed fields can be outlined and deserialized as a regular attribute. As discussed previously, attributes provide their own serialization and deserialization logic, in order to represent their data more concisely. The attribute can therefore contain any number of fields storing arbitrary data. In practice, serialization is handled automatically through automated JSON serialization, with no additional user-input being required. Internally, DGIR distinguishes typed- and untyped-attributes explicitly, where typed-attributes have to hold a type and additional storage fitting that type. Untyped-attributes may contain no data additional fields but the `ident` and can be used as markers.

Basic typed attributes may look like this:

```
1 {
2   "ident": "IntegerAttr",
3   "type": "uint8",
4   "value": 255
5 }
```

Listing 31: DGIR JSON IntegerAttribute.

```
1 {
2   "ident": "TypeAttr",
3   "type": "func.func<\"() -> ()\">"
4 }
```

Listing 32: DGIR JSON TypeAttribute.

**6.1.1.3 Values** Since DGIR does not use SSA-form, `Values` connect the output of one or multiple operations to the inputs of other operations. Values are immutable, typed objects representing the data-flow of the program and must be uniquely identifiable in the context of the whole program. Optionally, values may hold debug information mapping the value to its source name and location.

When encountering a value for the first time during serialization, they are serialized as an object using the syntax seen in Listing 33. Value objects must provide a unique identifier through the `@id` field, which is used during subsequent encounters to serialize a reference to the existing object.

Listing 34 shows how during the initial definition of the value `%s1_0` the full value object is serialized, while the subsequent redefinition(`str.concat`) simply serialize the textual identifier of the existing value. During deserialization, value objects will be encountered top to bottom in the same order they way serialized, therefore not requiring forward reference resolution or two pass deserialization.

```

1 {
2   "@id" : String,
3   "type" : String,
4   "debug"? : {
5     "loc" : String,
6     "name" : String
7   }
8 }

```

Listing 33: DGIR JSON Value syntax.

```

1 {
2   "ident" : "arith.constant",
3   "attributes" : [ {
4     "name" : "value",
5     "attr_ident" : "stringAttr",
6     "attr_type" : "string",
7     "attr_value" : "Hello, "
8   } ],
9   "output" : {
10    "@id" : "%s1_0",
11    "type" : "string"
12  }
13 }, {
14   "ident" : "arith.constant",
15   "attributes" : [ {
16     "name" : "value",
17     "attr_ident" : "stringAttr",
18     "attr_type" : "string",
19     "attr_value" : "world!"
20   } ],
21   "output" : {
22     "@id" : "%s2_1",
23     "type" : "string",
24     "debug" : {
25       "loc" : "valueDemo.java:4:12",
26       "name" : "s2"
27     }
28   }
29 }, {
30   "ident" : "str.concat",
31   "operands" : [ "%s1_0", "%s2_1" ],
32   "output" : "%s1_0"
33 }

```

Listing 34: DGIR JSON Value usage example.

The textual identifier of values must adhere to the following rules:

- The first character is always "%".
- Program wide, every value must have a unique textual identifier.
- Should a value have a debug name, that name will be prepended to the unique numeric identifier.

Using unique identifiers across the whole program avoids ambiguities that could arise from scoped naming rules. The optional debug data of values can be used to provide source mapped debug information about allocated storage. Additionally, serializing textual value IDs based on the original variable name, improves the readability of the IR substantially.

#### 6.1.1.4 Regions and Blocks DGIR programs are structured using two distinct concepts

1. **Block**: Semantically equivalent to CFG basic blocks, containing an ordered list of operations with a single entry and exit operation.
2. **Region**: An ordered list of blocks with a fixed entry block and an arbitrary amount of value inputs, that are visible from each contained block. These are semantically closest to scopes in high level languages.

Each operation contains a list of regions, with each region containing a list of blocks. With ops that are isolated from above, values from the containing region are not visible to the operations contained in its nested regions. When it encounters referenced values from outside isolated regions, the operation verifier will emit an error.

The syntax for regions and blocks looks can be seen in Listings 35 and 36.

```

1 {
2   "bodyValues"? : [ { Value }, ... ],
3   "blocks" : [ { Block }, ... ]
4 }

```

Listing 35: DGIR JSON Region syntax.

```

1 {
2   "@id" : String,
3   "operations" : [ { Operation }, ... ]
4 }

```

Listing 36: DGIR JSON Block syntax.

It is of the highest importance that the order of regions and the blocks within them, remains consistent during serialization and deserialization. Operation semantics rely on this order to determine control flow. Operations like the **IfOp** from the structured control flow dialect, require the first region to represent the *if*- and the second region to represent the *else*-branch. Furthermore, the entry point of a region is defined as the first operation in the first block. Therefore, changing the order of either could lead to incorrect program semantics.

Deserializing blocks is not as straightforward as other objects, since operations might reference block-ids of blocks not yet deserialized; for example when branching out of the entry block. For this reason DGIR splits operation deserialization into two phases. During the first phase we deserialize all blocks, populating branching operations with placeholder targets. Once the whole region containing the block is deserialized, a second pass resolves all textual references and updates the targets of all operations to point to the actual block.

**6.1.1.5 Types** As described above, all values must have a fixed type, which are used to enforce DGIR’s static typing rules. During validation, ops must ensure that the type of all their operands and result match the expected type signature. Types are serialized as textual expressions following the syntax from Listing 22, allowing the use of simple and parameterized types.

### 6.1.2 Textual IR Form

While the JSON form is the semantically most complete serialized form of DGIR and currently the only fully supported exchange format, programs written in that format tend to balloon in size quickly, with single operations spanning multiple lines. This can make it hard to read and debug serialized DGIR, as JSON’s structural object hierarchies obfuscate DGIR’s structural primitives. Bril faced the same problem, and therefore introduced a textual form that is a lot more compact and closer to LLVM IR than its JSON equivalent. For the same reasons, DGIR provides a custom textual format, stylistically somewhere between Bril’s textual format and MLIR.

Listing 53 gives an approximate syntax for DGIR’s textual representation, with Listings 37 and 38 providing a concrete example of a simple Java program and its textual DGIR equivalent. While DGIR’s textual representation contains most information required to reconstruct the original IR directly, doing so will not be possible without the contextual information of the specific op. Dynamic attributes can not be deserialized, since their specific type is specified at runtime and attribute declarations do not include their types in the textual IR. If the need arises to support full deserialization of the textual form, the attribute ident has to be added to the attribute declaration.

```

1 public class doLoop {
2     public static void main(int ignored1, float ignored2) {
3         int x = 0;
4         do {
5             x++;
6         } while (x < 5);
7         assert x == 5 : "Expected x to be 5, but got " + x;
8     }
9 }
10

```

Listing 37: A simple do while loop program implemented in Java.

### 6.1.3 Validation and Analysis

One of the central goals of DGIR was to support static validation of all operation, type, and storage semantics. For this purpose DGIR provides user-defined, as well as builtin validation support. For a program to be found valid, every op has to follow DGIR’s structural rules, governing control flow, data flow, and type safety. The generalized operation validator will assert these rules and reject incorrect programs.

A correct program has to fulfil the following requirements:

- Every value operand of every operation has to have a value assigned to it. Same goes for every block operand (successor).
- The backing attribute for each named attribute has to be set.
- The object stored in typed-attributes has to match the specified type.
- In case the default attributes(see Section 6.2) are populated with a specific attribute type, the operation has to hold an attribute of the same type. Furthermore, if the default attribute held a typed-attribute, the held typed-attribute must specify same type.
- Operations may not contain static attributes which are not part of the default attributes.
- Branching to the first block in a region is not allowed. This ensures that the CFG only has one entry point.
- Blocks must contain a terminator operation, and the terminator has to be the last operation in the block(Exception made for operations implementing trait `INoTerminator`).

```

1 func.func [symbol_name = {main}, type = {func.func<"() -> ()">}] @doLoop.java:2:3
2 {
3   .blk_0:
4     %x_0 :int32 = arith.constant [value = {0}] @doLoop.java:3:13
5     scf.scope @<ignore>:-1:-1
6     {
7       .blk_1:
8         scf.while @doLoop.java:4:5
9         {
10          .blk_2:
11            %x_0 :int32 = arith.unary %x_0 [unaryMode = {INCREMENT}] @doLoop.java:5:7
12            cf.br ==> .blk_3 @doLoop.java:4:5
13
14          .blk_3:
15            %1 :int32 = arith.constant [value = {5}] @doLoop.java:6:18
16            %2 :int1 = arith.bin %x_0, %1 [binMode = {LT}] @doLoop.java:6:14
17            cf.br_cond %2 ==> .blk_4, .blk_5 @<ignore>:-1:-1
18
19          .blk_4:
20            scf.continue @doLoop.java:6:14
21
22          .blk_5:
23            scf.end @doLoop.java:6:14
24        }
25      {
26        .blk_6:
27          scf.continue @doLoop.java:5:5
28      }
29      scf.end @<ignore>:-1:-1
30    }
31    %3 :int32 = arith.constant [value = {5}] @doLoop.java:7:17
32    %4 :int1 = arith.bin %x_0, %3 [binMode = {EQ}] @doLoop.java:7:12
33    %5 :string = arith.constant [value = {Expected x to be 5, but got }] @doLoop.java:7:21
34    %6 :string = str.concat %5, %x_0 @doLoop.java:7:21
35    cf.assert %4, %6 @doLoop.java:7:5
36    func.return @doLoop.java:8:5
37 }

```

Listing 38: Java program from Listing 37 compiled to DGIR textual form.

- Only terminators may branch.
- All operation successors have to target blocks in the same region.
- Used values have to be defined on all control flow paths leading from the entry block to the use-site. Usage of conditionally defined values is not permitted.

Verification of the use of values is done through reaching-definitions. We analyze the domination relationship between basic blocks and uses an iterative process to generate a set of incoming and outgoing value definitions for each basic block. Regions that are not attached to *isolated from above* operations inherit the values visible to the owning operations. They may also hold so called *region values* which are visible to all blocks inside a region, as they are semantically equivalent to values defined inside the entry block. A more detailed definition of the algorithm generating the reaching definitions is given in Listing 56.

The builtin analysis only verifies the general structural integrity of a program and cannot validate the semantics of the specific operations. Instead, any semantic analysis such as argument count and type, return types, attribute values, visibility, etc. have to be handled by the dialect implementations. During verification, the validation logic for all used types, operations and their implemented traits (Section 6.2.5) are invoked. If any of the validation reports an error, validation fails and the program is rejected. Thanks to op traits a lot of validation logic can be reused across different implementations.

**6.1.3.1 CFG Generation** DGIR provides additional tooling to extract the CFG from programs, in the form of dot graphs. The dot graph for Listing 38 can be seen in Fig. 9.

#### 6.1.4 Implemented Extension Surface

DGIR uses the concept of dialects to provide its extension surface. *Dialects* provide the semantic grouping of domain-specific operations, types, and attributes, organizing them under a collective namespace. Since DGIR is based largely around the concept of type-erased descriptor classes, every operation, type, and attribute has to provide a fixed set of factories and accessors providing the type-erased functionality. Registering a dialect makes it collect all its declared elements and store their details in a global dialect-registry (*DGIRContext*). In order to ensure a unified API surface for the descriptors providing the details, DGIR provides a set of contracts in the form of abstract base classes, describing the required contents of all dialect elements. Dialects can be loaded at runtime and do not have to be part of the same binary as DGIR.

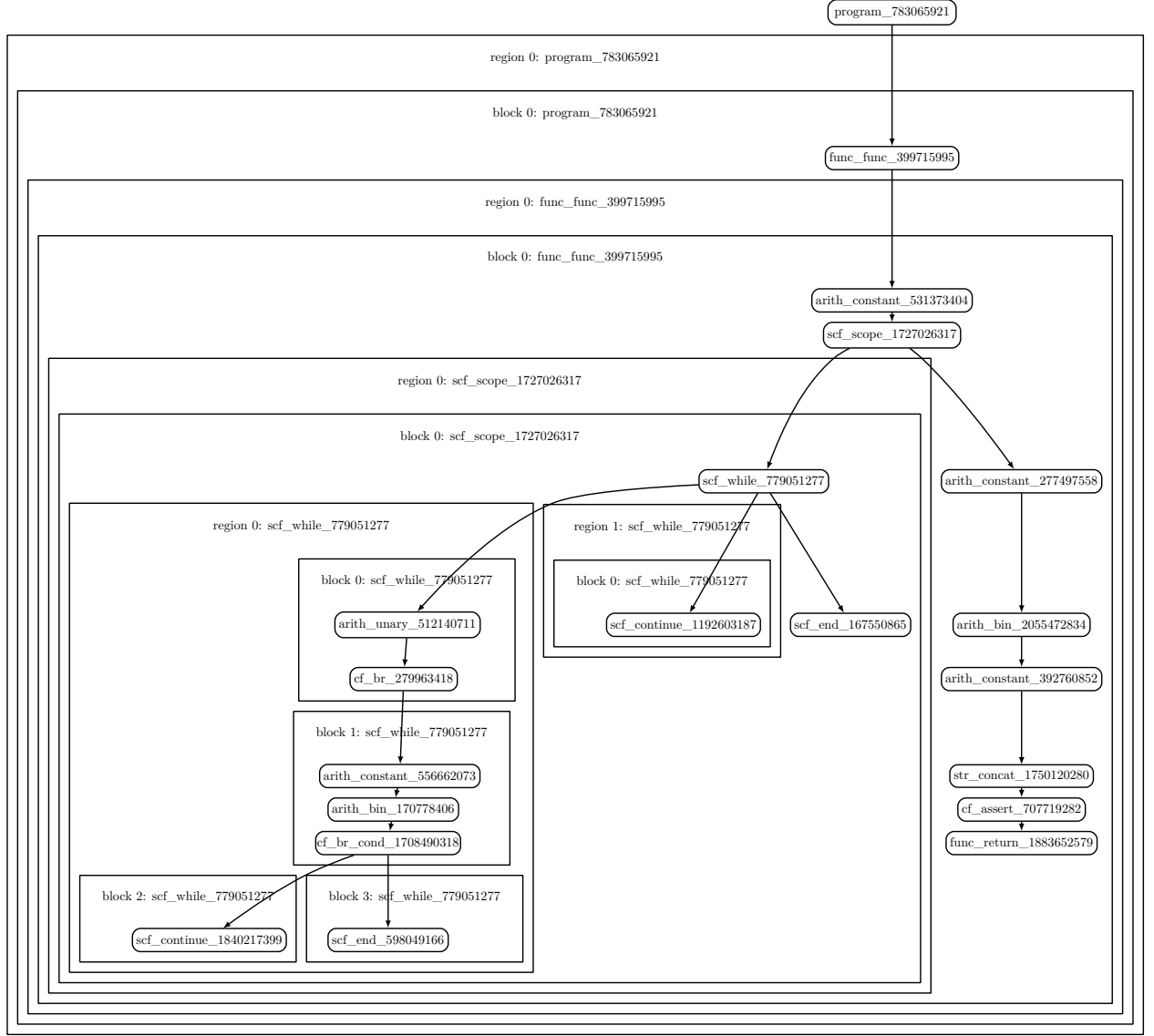


Figure 9: CFG for DGIR program in Listing 38.

The op-class provides the description and semantics of domain-specific operations. This class serves both as the semantic wrapper around the underlying operation, as well as the descriptor class used to register the type-erased representation of the op in the dialect registry. The op description includes static properties of the operation, like its inherent attribute names and types, or the factory for creation of the op from an operation. Ops do not hold any data apart from their operation reference, providing contextual accessors to the data of their operation.

DGIR uses types to describe what kind of data a value holds. Just like ops, types also represent arbitrary semantics, defined by their dialect. At the core of DGIR sits a strict type system, which enforces that operations may only redefine values with the exact same type. Types are considered equal when their identity(pointer) is equal, which is enforced through a uniquing process, in which two types are considered to be the same when their parametric string representation is equal. To avoid type errors in previously valid value definitions, value types are immutable. Types-details such as its identifier and parser, are provided through an external descriptor object and associated with new type instances during construction.

The last element extensions can provide are attributes, in either typed or untyped form. Attributes provide the external storage needed by operations like constants, to represent their semantics fully. In the case of typed-attributes this storage is accompanied by a DGIR type and the storage of the attribute is verified through the type's verifier. Attributes may hold arbitrary data.

### 6.1.5 Interpreter Workflow

DGIR was designed to be interpreter friendly, enabling deployment to a common, portable execution-environment. For this reason, DGIR ships with a custom interpreter, hooking into the registration system of DGIR, linking op-classes to a domain-specific execution handler. Its execution model is based on that of [brili](#), the TypeScript-based interpreter that comes with Bril. DGIR's interpreter uses an action based design, which invokes the domain-specific execution handler, and mutates the execution state based on the returned action-code. Since DGIR does not use a linear IR structure like Bril, branching and jumping cannot be accomplished via a regular program counter and must instead target blocks, regions and operations directly. The existing centralized dialect registry and abstraction mechanisms made the implementation of the interpreter and its extension system very straightforward.

Interpreter extensions follow the same grouping semantics as dialects, providing a dialect runner which contains multiple op runners. Dialect runners register all the op runners in a global lookup table, so that the interpreter can fetch the appropriate implementation at runtime. Every op runner has to provide the class it is targeting, as well as an implementation providing the unified execution hook with the signature

```
Action runImpl(Operation op, State state);
```

The state gets passed in by the VM and allows the runner to access the storage bound to values, in addition to opening and closing scopes. Once the hook has finished execution, the interpreter loop will continue execution based on the semantics of the returned action, the definitions of which can be seen in Listing 39. Most of the actions trigger common behaviour like advancing a single operation, calling a function or jumping to a label. Because of DGIR's region semantics, two additional actions are required: [JumpToRegion](#) which allows execution to continue in another region of the same operation, and [StepIntoRegion](#) which forwards execution to the entry of a given child region of the current operation. These are required, since regions form a semantic scope, with limited visibility to the outside program. For example, in the case of [WhileOps](#), two regions are required to cleanly separate the condition and body evaluation, with control flow passing between the two, until the condition is false. Here the [JumpToRegion](#) action is utilized, so that execution does not have to exit the semantic scope of the [WhileOp](#).

```
1  /* Executes the next operation in the current block. */
2  record Next(){}
3  /* Jumps to the given block and executes it. The block must be in the same region as the current block.*/
4  record JumpToBlock(Block target) implements Action {}
5  /* Jumps to the given region and executes it. The region must be part of the same operation as the current operation. */
6  record JumpToRegion(Region target, List<Object> args){}
7  /* Steps into the given region. */
8  record StepIntoRegion(Region region, boolean isolatedFromAbove, List<Object> args){}
9  /* Calls the given function operation. */
10 record Call(Operation funcOp, List<Object> args){}
11 /* Terminates the current block and returns the given value. The value can be null if the block does not return anything. */
12 record Terminate(Object value, boolean fromCall){}
13 /* Aborts the execution of the program with the given message. */
14 record Abort(String message, Optional<Exception> e){}
```

Listing 39: Interpreter actions for the DGIR interpreter.

To execute a DGIR program, first all dialects and their dialect runners have to be registered, before loading the program itself. After an initial validation, execution can be started with or without a debugger attached. DGIR's interpreter implementation provides a DAP enabled debugger, which can be used to perform breakpoint and step debugging, and variable inspection using modern code editors.

## 6.2 API Complexity

DGIR's final implementation exposes a compact and explicit extension surface for defining dialects similar to MLIR, but with greatly reduced API and conceptual complexity. Compared to MLIR, DGIR offers significantly reduced flexibility, which reduces the amount of boilerplate required to support DGIR's whole feature-set. While DGIR requires dialects to provide a similar amount of metadata for its elements, it exposes the required accessors more explicitly and strives for improved locality and reuse, over runtime performance.

However, MLIR does provide automated code generation through so called *table-gen* gen files, which allows authors to provide definitions of all types in a highly concise manner. During compilation, the C++ source files are generated from their *table-gen* descriptions. Using *table-gen*, authors can describe additional constructors, a custom assembly-format, member names, accessors and validation, and more. For advanced use-cases, members of the generated classes can be declared without implementations, while developers provide the definition of those members directly through in C++. Alternatively inline C++ may also be used. In practice, MLIR *table-gen* is mostly used to avoid the tedious implementation of getters and setters, assembly formats and to describe the general shape of operations, while the more complex behavior or custom assembly formats are implemented

in C++. Since the storage semantics of MLIR are so complex, to stay accessible, source generation is almost unavoidable, which is proven by the fact that currently only two ops in MLIR’s ecosystem are written without the use of code generation([AffineDmaStartOp](#) and [AffineDmaWaitOp](#)).

Listing 61 demonstrates why this workflow is required, even for conceptually simple operations like MLIR’s [ConstantOp](#). MLIR’s class declaration alone, without most of the method implementations, requires more than double the amount of lines of the full DGIR implementation of [ConstantOp](#) (Listing 62). Comparing the two implementations clearly demonstrates the conceptual differences between the two implementations, with MLIR providing much greater flexibility compared to DGIR, at the cost of verbosity. However, in its listed form, DGIR’s implementation does lack the validation necessary to guarantee that [ConstantOps](#) provide the correct return type for their attribute storage.

Additional comparisons between MLIR’s table-gen/C++ source-code and DGIR would extend beyond the scope of this work and will therefore be excluded from the following discussions.

Since DGIR’s memory and construction model is so much simpler compared to MLIR, the use of tools like code generation can be fully avoided without elevating the barrier to entry. The only area where DGIR falls significantly short of MLIR’s table-gen approach, is its lack of automated printer and parser generation for custom type syntaxes. To provide evidence to our claim of DGIR’s reduced API complexity, we will now discuss the required steps to implement a new dialect containing operations, types and attributes.

### 6.2.1 Dialect

Registering elements of a given dialect with the DGIR context is accomplished through an object extending the abstract dialect class. Dialects must implement the provider methods [allOperations](#), [allTypes](#) and [allAttributes](#). Calling the dialect’s [register](#) method triggers the collection and registration of all elements provided by these methods. Dialect’s sole purpose is the management and grouping of semantically related elements under a common namespace. Listing 40 demonstrates the concrete implementation of a dialect containing arithmetic operations. Dialects may choose to provide any number of ops, types, or attributes, but are not required to do so. Empty dialects or one providing only a single element type are completely valid.

```

1 public class ArithDialect extends Dialect {
2     @Override
3     public String getNamespace() {
4         return "arith";
5     }
6
7     @Override
8     public List<Op> allOps() {
9         return List.of(new ConstantOp(), new BinaryOp(), new ArithmeticOp(), new CastOp());
10    }
11
12    @Override
13    public List<TypeDescriptor> allTypes() {
14        return List.of();
15    }
16
17    @Override
18    public List<AttributeDescriptor> allAttributes() {
19        return list.of(new UnaryModeDescriptor(), new BinModeDescriptor());
20    }
21 }

```

Listing 40: Exemplary implementation of the arithmetic-dialect class.

It is the responsibility of the author to call the dialect’s registration method before their elements are instantiated. Instantiating dialect elements before they are registered will cause an exception to be thrown, as DGIR does not support the use of opaque(unregistered) operations, types, or attributes.

### 6.2.2 Op

Domain-specific operations are represented by classes inheriting from the abstract base-class op. Every concrete op has to define five methods:

- `Class<? extends Dialect> getDialect():` Returns the class of the dialect this op belongs to.
- `String getIdent():` Returns the unique identifier of the operation. Must be prefixed with the dialect namespace.
- `Supplier<List<NamedAttribute>> defaultAttributes():` Returns a supplier which produces the list of attributes that are inherent to the op(cannot be added or removed after creation)

- `Function<Operation, Boolean> getVerifier()`: Returns the semantic validator for operations of this type.
- `Function<Operation, Op> getOpFactory()`: Returns a factory which wraps the given generic operation in its semantic op wrapper. Used during deserialization and when casting from operations to ops using the operation details (Listing 24)

Since ops serve as a layer of abstraction themselves, DGIR handles op-metadata registration and op semantics through the same class. This is in contrast to types and attributes which have to provide separate descriptor objects. Implemented op traits will be queried through reflection and therefore do not need to be supplied explicitly.

Listing 62 demonstrates how DGIR’s `ConstantOp` is defined, demonstrating the use of default attributes for the value storage, op traits for reusable abstraction and validation (`INoOperands`, `IHasResult`) and op-construction through factory hooks.

### 6.2.3 Type

To define types for DGIR, dialects have to provide two classes per definition: the semantic type with parametric storage and the type descriptor providing the static type information. Parametric types need to provide their own instance specific serialization logic, while the type descriptor provides the means to deserialize the custom type syntax into specific instances. Since all static type information lives in the type-descriptor class, simple, non-parametric types mostly do not need to specialize any of the semantic type’s default behaviour. Type descriptors on the other hand have to provide detailed type information, similar to ops.

Therefore, every type descriptor needs to implement the following methods:

- `Class<? extends Dialect> getDialect()`: Returns the class of the dialect this type belongs to.
- `Class<? extends Type> getTypeClass()`: Return the semantic type class that is being described.
- `String getIdent()`: Returns the unique identifier of the type. Must be prefixed with the dialect namespace and not type parameters.
- `Function<Object, Boolean> getValidator()` Returns a function that checks whether a given value is a valid instance of this type.
- `Function<Pair<String, TypeDetails>, Type> getParameterizedIdFactory()` Returns a factory that creates a type from a parameterized identifier. Non-parametric types should return the unique instance belonging to the given identifier.
- `void initDefaultTypeInstances()` Called after registration of the type. Should be used to initialize default type instances(e.g. `int8`, `int16`, etc.) after their details are made available. Otherwise, the type constructor will throw; opaque(unregistered) types are not allowed.

To ensure that all type instances sharing the same parametric signature can be compared efficiently(without having to compare their string representation), DGIR provides a type uniquer. The type uniquer takes an instance of a type and returns the unique instance for the given signature, by looking up the parametric representation in a global registry. Parametric types should therefore define a static factory, taking the type parameters and returning the unique type instance. Non-parametric types can simply expose the instances created by `initDefaultTypeInstances()` (Listing 41).

```

1 // Function type unique factory
2 public static FuncType of(List<Type> inputs, Type output) {
3     return TypeUniquer.uniqueInstance(new FuncType(inputs, output));
4 }
5 // Integer type unique default instance
6 static IntegerT int8 = null; // Initialized by IntegerTypeDescriptor after registration
7 public static IntegerT INT8(){
8     return Objects.requireNonNull(int8);
9 }

```

Listing 41: Type instance uniquing example.

The full implementation of a complex type with custom parametric syntax can be seen in Listing 59, where both the definition of the `func.func` type and its descriptor are listed.

### 6.2.4 Attribute

Attributes follow the same general declaration syntax as types, where attributes are split into a semantic and a descriptive portion. However, the semantics of attributes are much simpler as their only purpose is to attach

external storage to operations. For this reason, attribute details mainly serve as an indirection to look up the appropriate deserializer for a given ident, consequently requiring the smallest descriptor of all DGIR elements.

Attribute descriptors need to implement the following methods:

- `Class<? extends Dialect> getDialect()`: Returns the class of the dialect this attribute belongs to.
- `Class<? extends Attribute> getAttributeClass()`: Return the semantic attribute class that is being described.
- `String getIdent()`: Returns the unique identifier of the attribute. Must be prefixed with the dialect namespace.

Listing 42 gives the definition of the descriptor for the `StringAttribute`, which is part of the `StrDialect` and has the ident `str.attr`.

```
1 class StringAttributeDescriptor implements StrAttrDescriptor {
2     default Class<? extends Dialect> getDialect() { return StrDialect.class; }
3     public Class<? extends Attribute> getAttributeClass() { return StringAttribute.class; }
4     public String getIdent() { return "str.attr"; }
5 }
```

Listing 42: Implementation of string attribute descriptor.

Attributes themselves need to implement the `Object getStorage()` method, returning the complete storage of the attribute, bundled into one object. Textual serialization relies on this method. Since JSON serialization of simple objects such as attributes can be fully automated, no explicit serialization and deserialization logic needs to be provided. To ensure frictionless deserialization and flexible storage semantics, attribute storage should be fully mutable. Implementing storage attributes like `StringAttribute` (Listing 43) is as simple as defining a class inheriting the `TypedAttribute` class, adding a storage member for the string and returning the value of the storage in the `getStorage` method. No serialization annotation or code required.

```
1 class StringAttribute extends TypedAttribute {
2     private String value;
3     public StringAttribute(String value) {
4         super(StringT.INSTANCE());
5         this.value = value;
6     }
7     public Object getStorage() { return value; }
8     public String getValue() { return value; }
9     public void setValue(String value) { this.value = value; }
10 }
```

Listing 43: Implementation of string attribute.

### 6.2.5 OpTrait

Op traits are interfaces representing reusable abstractions and validation for operations. Every op trait must extend the minimal `IOpTrait` interface, which provides an accessor for the operation of the implementing op via `Operation getOperation()`. This accessor is part of the op class, therefore requiring no further definitions by the author. An op trait may be used to provide any type of accessor and should do so through default implemented interface-methods. During registrations, all implemented op trait classes of a given operation are stored in the operation details, making it possible to query their implementation without accessing the defining op. Further, every op trait needs to provide a verification method with the signature `static boolean verify(Operation operation)`, providing the semantic validation for the trait.

Listing 44 gives an example `OpTrait` implemented in DGIR; the `ISingleBlock` trait, which ensures an operation has exactly one block, and provides additional accessors relying on this assumption. Every op that should inherit this semantic property may implement the trait, skipping manual validation. Traits like `IIsolatedFromAbove` are used during verification to ensure the program is structurally sound.

## 6.3 IR Simplicity

DGIR's textual form, retains most of Bril's simplicity, while incorporating MLIR's more flexible extension system and hierarchical control-flow structure through regions. Supporting the additional extension features does however make the JSON IR format more verbose, increasing the JSON complexity significantly (see Listing 54). Compared to MLIR, DGIR's extension mechanism is more streamlined with a minimal API surface. To achieve this simplified implementation, DGIR does not have to resort to helpers like source-generation, or opaque strategies based on reflections. Instead, the decision was made to remove a large portion of the most complex features supported by MLIR, merging functionality into simpler features where possible. As stated above,

```

1 public interface ISingleBlock extends IOpTrait {
2     static boolean verify(Operation operation) {
3         if (operation.getRegions().size() != 1) {
4             operation.emitError("Operation must have exactly one region.");
5             return false;
6         }
7         if (operation.getFirstRegion().orElseThrow().getBlocks().size() != 1) {
8             operation.emitError("Operation's single region must have exactly one block.");
9             return false;
10        }
11        return true;
12    }
13    default Block getBlock() {
14        return getOperation().getFirstRegion().orElseThrow().getEntryBlock();
15    }
16    default Operation addOperation(Operation operation) {
17        getOperation().getFirstRegion().orElseThrow().getEntryBlock().addOperation(operation);
18        return operation;
19    }
20    default <OpT extends Op> OpT addOperation(OpT op) {
21        getOperation().getFirstRegion().orElseThrow().getEntryBlock().addOperation(op);
22        return op;
23    }
24 }

```

Listing 44: Implementation of the single block OpTrait.

maintaining MLIR’s extension mechanism, its user-facing API and its memory model, is only made possible through its extensive use of table-gen. It provides the necessary abstraction and automation that is required to maintain and implement all components required to author a dialect.

Bril on the other hand does not offer a native extension mechanism, instead relying on direct source modifications for every extension. However, Bril’s source code is kept simple and highly concise, allowing such modifications to be implemented with relative ease. Extending the operation or type system requires minimal changes to the IR itself, with the bulk of the implementation complexity arising from interpreter, serialization, and validation support.

### 6.3.1 Regularity

Due to its hierarchical, recursive nature, DGIR’s core syntax is not formally regular, while Bril is. Instead, DGIR programs can be modelled through a context-free grammar, as all objects are represented using the same generalized syntax. While DGIR is not regular, one could argue that it is more consistent than Bril, since all possible semantics are modelled through the same basic operation structure. Bril on the other hand bases its IR form on multiple components (functions and instructions contained in them) which are represented by multiple distinct syntactic objects.

```

1 void main(){
2     int a = 47;
3     int b = 42;
4     int c;
5     if (true){
6         b = 1;
7         c = 5;
8     }else{
9         a = 2;
10        c = 10
11    }
12    int d = a - c;
13    printf("%d", d);
14 }

```

Listing 45: Simple conditional program written in C.

To get a better understanding of the differences between DGIR, Bril and MLIR, we will compare each textual IR form of the high level program in Listing 45. The comparison will be done using the textual form of both DGIR and Bril, as the equivalent JSON form is significantly less readable and compact.

Listing 46 compares Bril’s and DGIR’s representation of the program using the same semantic concepts of functions, basic blocks, and branching operations. DGIR’s form clearly demonstrates the basic conceptual differences between the two IRs, where DGIR relies on fully defined basic blocks (`.entry`), with terminators at the end of each (`func.return`) This stands in contrast to Bril’s more implicit control flow behavior, which allows authors to omit the entry point of a function, and, in case of void return types, also the return statement. Further, DGIR encodes its constant storage using named attributes, which are significantly more verbose than Bril’s constants. However, even including these additional elements does not increase the length of the program

significantly, as all Bril instructions can be translated directly into equivalent operations. Using DGIR’s abstraction capabilities and replacing the raw control flow with structured control flow in the form of an if-statement, reduces the lines of codes further, while expressing the code semantics more explicitly (Listing 47). While their textual formats are comparable in complexity, DGIR’s JSON format is a lot more complex, due to its reliance on nested, hierarchical structures. For the concrete example of Listing 46 see Listing 54.

|                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1 @main { 2   a: int = const 47; 3   b: int = const 42; 4   cond: bool = const true; 5   br cond .left .right; 6 7 .left: 8   b: int = const 1; 9   c: int = const 5; 10  jmp .end; 11 12 .right: 13   a: int = const 2; 14   c: int = const 10; 15  jmp .end; 16 17 .end: 18   d: int = sub a c; 19   print d; 20 }</pre> | <pre> 1 func.func [symbol_name = {main}, type = {func.func&lt;"() -&gt;   ↳ ()"&gt;}] 2 { 3   .entry: 4     %a: int64 = arith.constant [value = {47}] 5     %b: int64 = arith.constant [value = {41}] 6     %cond: int1 = arith.constant [value = {true}] 7     cf.br_cond %cond ==&gt; .left, .right 8 9   .left: 10    %b: int64 = arith.constant [value = {1}] 11    %c: int64 = arith.constant [value = {5}] 12    cf.br ==&gt; .end 13 14   .right: 15    %a: int64 = arith.constant [value = {2}] 16    %c: int64 = arith.constant [value = {10}] 17    cf.br ==&gt; .end 18 19   .end: 20    %d: int64 = arith.bin %a, %c [binMode = {SUB}] 21    io.print %d 22    func.return 23 }</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Listing 46: A simple conditional program in Bril (left) and DGIR (right) textual form.

```

1 func.func [symbol_name = {main}, type = {func.func<"() -> ()">}]
2 {
3   .entry
4     %a: int64 = arith.constant [value = {47}]
5     %b: int64 = arith.constant [value = {41}]
6     %c: int64 = arith.constant [value = {0}] // Needs to be declared upfront
7     %cond: int1 = arith.constant [value = {true}]
8     scf.if %cond
9     {
10      %b: int64 = arith.constant [value = {1}]
11      %c: int64 = arith.constant [value = {5}]
12    }
13    {
14      %a: int64 = arith.constant [value = {2}]
15      %c: int64 = arith.constant [value = {10}]
16    }
17    %d: int64 = arith.bin %a, %c [binMode = {SUB}]
18    io.print %d
19    func.return
20 }
```

Listing 47: Abstracted DGIR program from Listing 46

MLIR’s generic textual form is comparably complex to that of DGIR while being a little more verbose, with the most notable difference being the location of type annotations, which are provided through an additional function type signature at the end of each MLIR operation. DGIR uses infix type annotations in the style of Bril. Readability of region-holding operations like `func.func` is further reduced by the fact that attributes containing the method name and type, are moved behind the body-region. MLIR’s reliance on block arguments for its SSA semantics, adds additional visual noise, with branching operations having to pass their arguments along explicitly.

Using MLIR’s custom assembly formats reduces the verbosity and complexity of the final output drastically, bringing it inline with Bril. MLIR allows the first block in a region to be declared implicitly as branching to that block is forbidden. The resulting IR using both the generic and custom assembly formats can be seen in Listing 48.

## 6.4 Extensibility in Practice

DGIR evolved alongside its interpreter and a Java compiler frontend, used to compile dungeon-specific programs to its domain-specific dialect. Additionally, MLIR provides multiple dialects for commonly required operations, such as arithmetics, structured and regular control-flow, console in- and output and array operations. Most ops

```

1 "builtin.module"() ({
2   "func.func"() ({
3     ^bb0:
4     %a = "arith.constant"() {value = 47 : i64} : () -> i64
5     %b = "arith.constant"() {value = 42 : i64} : () -> i64
6     %cond = "arith.constant"() {value = true : i1} : () -> i1
7     "cf.cond_br"(%cond)[^left, ^right] : (i1) -> ()
8
9     ^left:
10    %b_left = "arith.constant"() {value = 1 : i64} : () ->
11    ↪ i64
12    %c_left = "arith.constant"() {value = 5 : i64} : () ->
13    ↪ i64
14    "cf.br"(%a, %c_left)[^end] : (i64, i64) -> ()
15
16    ^right:
17    %a_right = "arith.constant"() {value = 2 : i64} : () ->
18    ↪ i64
19    %c_right = "arith.constant"() {value = 10 : i64} : () ->
20    ↪ i64
21    "cf.br"(%a_right, %c_right)[^end] : (i64, i64) -> ()
22
23    ^end(%a_end: i64, %c_end: i64):
24    %d = "arith.subi"(%a_end, %c_end) : (i64, i64) -> i64
25    "io.print"(%d) : (i64) -> ()
26    "func.return"() : () -> ()
27  }) {sym_name = "main", function_type = () -> ()} : () -> ()
28 }) : () -> ()

```

```

1 module {
2   func.func @main() {
3     %a = arith.constant 47 : i64
4     %b = arith.constant 41 : i64
5     %cond = arith.constant true : i1
6     cf.cond_br %cond, ^left, ^right
7
8     ^left:
9     %b_left = arith.constant 1 : i64
10    %c_left = arith.constant 5 : i64
11    cf.br ^end(%a, %c_left : i64, i64)
12
13    ^right:
14    %a_right = arith.constant 2 : i64
15    %c_right = arith.constant 10 : i64
16    cf.br ^end(%a_right, %c_right : i64, i64)
17
18    ^end(%a_end: i64, %c_end: i64):
19    %d = arith.subi %a_end, %c_end : i64
20    io.print %d : i64
21    func.return
22  }
23 }

```

Listing 48: MLIR form of program in Listing 46. Generalized printer(left) and customized(right).

are extensively tested in isolation using unit tests, with the Java compiler providing the necessary integration tests for real world usage.

Throughout the implementation process, DGIR’s descriptor-backed extension mechanism proved to be capable of modeling arbitrary computational needs simply and efficiently. The integration tasks provided valuable feedback for the API design of DGIR, with operation semantics evolving with the required functionality. Simultaneously, DGIR’s structural verification mechanisms provided valuable feedback for the compilation and interpreter implementation. Since the interpreter can rely on the guarantees provided by the structural analysis, runtime validation of the executed code can be reduced to a minimum. One example of this is value liveness, which can be guaranteed statically through its reaching-definitions analysis. Similarly, the compiler does not have to check for value liveness during compilation as it can defer this analysis to the IR itself.

An example for where compiler integration helped identify weaknesses of a specific dialect’s semantics, was during compilation of the ternary conditional operator. The goal was to model the operator using an `IfOp`, which initially could not return values from its branch regions. This was a problem since ternary operations can return values of any type and forward declaring an uninitialized value is not a feature supported by DGIR. This means that the value would have to be defined beforehand with a typed default. To avoid this, `IfOp`’s semantics were extended to allow yielding values from the branch bodies. Thanks to DGIR’s straightforward API, implementing these changes only a couple dozen line changes, even though it required the introduction of a new op and changes to the validation of multiple others.

Similarly, implementing the interpreter revealed multiple areas where generalized validation opportunities could improve reliability and reduce runtime validation overhead. One such opportunity was to unify the validation of parent-child relationships, where the use of some operations is only valid if their containing regions belong to a specific operation-type. This property is important for terminals like function returns, which are only allowed as children of `FuncOps`, as returning directly from nested regions would cause unpredictable control-flow implications. To generalize this validation step the `ISpecificParentOp-OpTrait` was introduced, which requires the implementing op to provide all valid parent types, generalizing the required validation for all implementers.

## 6.5 Design Trade-offs and Non-goals

While DGIR supports a large extension feature set and ships with a capable library of builtin validation and verification algorithms, many features of the project that DGIR bases its design on are omitted on purpose. The final design of DGIR omits optimization-oriented infrastructure and retains an analysis- and interpretation-oriented core. In practice this means that DGIR is not sufficiently equipped to support advanced optimization techniques based on IR rewrites. The main reason for this lack of capability is DGIR’s design centered around non-SSA values, which makes reasoning about data flow a lot more complicated. While IR transformations

are possible and reasonably simple through its eagerly maintained use- and def-chains, no generalized rewrite system, like MLIR’s pattern matching, is available. DGIR also leaves out any builtin optimizations, as its focus lies on execution in a debugging environment, where optimizations may interfere with the debugging experience.

Instead, DGIR focuses its design on areas like verification and validation, interpretation-oriented workflows, API ease of use, and simple value semantics. However, DGIR’s target interpretation workflows do not aim to provide a performance-oriented design, instead focusing on ease of use, abstraction support, and native extensibility.

## 6.6 Summary of Findings

This chapter reported the finished DGIR artifact and organized its outcomes based on the research question. The reported outcomes are that DGIR features a consistently lower API complexity compared to MLIR, while supporting significantly greater structural flexibility than Bril. DGIR proved its sufficient robustness for compiler-frontend and interpreter integration through practical integration tasks, providing an intentionally performance-neutral, non-optimization-centered design.

Section 7 evaluates the success of these outcomes against the research question and discusses limitations and implications.

## 7 Discussion

This chapter evaluates the success of the project relative to the research question and methodological objectives. The central research question asked how Bril’s simplicity can be combined with MLIR’s hierarchical and flexible design. Based on the results in Section 6, the project can be considered successful in the intended scope:

- It preserves a compact and explicit API compared to MLIR.
- It extends beyond Bril’s representational limits through regions and dialect-style extensibility.
- It remains robust enough for integration in a compiler frontend and interpreter workflow.

At the same time, this success should be interpreted as scope-bounded since the goal was not to match MLIR’s full optimization ecosystem. Instead, the goal was to provide a simplified development experience, providing an API that is more streamlined and concise, while preserving the general high-level abstraction capabilities of MLIR.

### 7.1 Strengths of the Final Design

The strongest outcome is the achieved middle ground between educational simplicity and practical extensibility. DGIR’s success at bridging the gap between Bril’s ease of use and MLIR’s versatility was demonstrated through its usage in a Java compiler frontend and its integration in the accompanying retargetable interpreter.

The final design provides:

- A consistent IR form that remains simple and portable.
- Explicit extension contracts through descriptors and operation wrappers.
- Robust construction and validation paths without requiring high-complexity infrastructure.
- Support for direct interpretation, with advanced debugging support through source mapping.

This aligns with the success criteria defined in Section 5 and supports the performance-neutral design stance of the thesis.

### 7.2 Limitations of the Non-SSA Choice

The decision against using SSA-form lowers implementation and usage complexity, but also imposes analytical limits. In particular, optimization patterns that rely on strict single-definition def-use chains become more difficult to express globally. While use-lists allow practical local rewrites, global reasoning for certain dataflow-dependent optimizations is less direct than in SSA-based systems. Adding support for SSA-form programs would enable the adoption of more advanced optimization techniques.

For the present thesis scope, this trade-off is acceptable because optimization power was not a primary objective. For future optimization-centric extensions, this area remains a clear limitation. However, incorporating optional SSA-enabled workflows would be possible through SSA-specific operations. Similarly to Bril, which does not rely on SSA form by default but adds support for it through  $\phi$ -like operations (Listing 49), DGIR could pursue a similar implementation style. Adopting Bril’s SSA style would require the implementation of a dialect, providing the SSA operations `get` and `set`. Since DGIR’s value semantics already support value serialization at any point, forward declaring the assignment target would not be a problem.

```
1  a: int = const 5;  
2  set c a;  
3  br cond .here .there;  
4  .here:  
5  b: int = const 7;  
6  set c b;  
7  .there:  
8  c: int = get;  
9  print c;
```

Listing 49: Bril program using SSA form[28].

To fully support SSA, a dataflow model would need to be defined in which SSA values can be passed into and out of regions. This is no trivial task as it has significant implications on memory boundaries and control flow. To address this problem, MLIR does not allow SSA values to be passed out of regions at all, instead relying on return values provided by terminator operations. Since DGIR does not support return multiple values from operations, a different solution or a changed operation structure is required. Either way, adding support would require further research.

### 7.3 Complexity Versus Expressivity

Adopting an MLIR-inspired structure introduces additional concepts compared to Bril, but this increase is justified by the expressive gains. Regions, descriptor-backed extensibility, and structured operation semantics enable representations that are difficult to encode cleanly in Bril’s flatter model.

The key discussion point is therefore not whether complexity increased at all, but whether complexity increased proportionally to expressive benefit. Given the integration outcomes reported in Section 6, the increase appears proportionate and aligned with project goals. While DGIR cannot provide an API as minimalistic and simple as Bril, extensive simplifications of MLIR’s extension mechanisms make DGIR significantly more accessible.

Currently, DGIR’s lack of runtime polymorphism is a major limiting factor when it comes to extensions that want to provide global analytical or transformative capabilities. Generalized abstractions can only be applied to dialects at compile time, complicating their implementation in respect to existing operations.

### 7.4 Automated Implementations

A key area in which MLIR’s table-gen based workflow offers significant advantages over DGIR is the declaration of custom assembly formats for all dialect elements. Table-gen assembly formats allow authors to skip the tedious implementation work of printers and parsers. While DGIR does provide authors with helpers to implement the serialization and parsing consistently, more manual work is required in comparison. Adding a more generalized implementation helper, or even automated printer and parser generation based on syntax descriptions would provide a significant improvement to DGIR’s current workflow.

Another area where DGIR is currently lacking is its type system and structural analysis of custom operations. Validation of semantic op-properties like argument count, layout and types, is the responsibility of the op’s author and needs to be reimplemented for every op. MLIR suffers from the same limitations, but thanks to table-gen, argument validation can often be fully, or at least partially, implemented automatically. This makes the validation more consistent, reliable and ensures it is always up to date with the current implementation of an op, making this another area where automated source generation would greatly improve the developer experience.

However, thanks to Java’s annotation processing capabilities, source generation could be supported as a native feature, removing the need for an external declaration language. Doing so would allow authors to declare custom syntax descriptions as part of their Java code, instead of splitting the implementation into multiple parallel files.

### 7.5 Conversion and Representation Limits

In Section 2.1.5, RKO’s language conversion capabilities and the general usefulness of being able to convert between different programming languages were discussed. Since DGIR will be used in a similar environment, source conversion based on IR output may become a relevant as well. In its current form, conversion from DGIR to high-level source languages can preserve semantics while still changing concrete syntax. This reflects a general property of decompilation and representation translation: semantic equivalence does not guarantee syntactic equivalence.

This means that while a program can be decompiled from DGIR, the resulting source might not feature the same syntactic elements as the original source file. Every syntactic element not supported by DGIR will be lowered to a semantically equivalent construct. One example of this is how Java switch-expressions are compiled to DGIR. Since Java switches can contain arbitrary computations in the case labels, be used as value-producing expressions, and declare their cases out of order, DGIR represents switches as nested if-else statements (Section A). Decompiling these would produce the same semantics as the original code but not the same syntax.

Richer metadata annotations could improve reconstruction quality for selected workflows. However, even with additional metadata, some representational loss remains unavoidable in general-purpose decompilation.

### 7.6 Overall Assessment and Implications

Overall, the project demonstrates that a practical hybrid of Bril-inspired simplicity and MLIR-inspired structure is feasible. The resulting system does not attempt to replace MLIR; instead, it defines a narrower and more accessible design point for educational and prototyping use cases. This thesis contributes a concrete IR architecture that keeps key hierarchical features while reducing high-complexity infrastructure.

## 8 Conclusion

This thesis set out to design, implement, and evaluate a portable, tooling-agnostic and accessible IR for the Probus project and related educational toolchains. The central objective of this thesis was to provide a common IR that can be targeted by multiple language frontends without inheriting the complexity of a full industrial compiler framework like MLIR. This goal was shaped by the needs of the Probus project, which requires a shared execution model for educational programming tasks, while remaining accessible and practical for student developers.

The design requirements derived from this context were:

1. Capture the semantics required by the Probus execution model and the game-specific operations used by the project.
2. Keep the IR modular and extensible so new language frontends and analysis features can be added without redesigning the core representation.
3. Preserve the educational focus by keeping the IR and its API readable, explicit and approachable.
4. Support integration into the surrounding tooling stack and verify the design through concrete implementation artifacts.

In order to arrive at the final design and implementation of DGIR, the thesis was split into multiple distinct steps. First, the work established the design space by comparing educational environments and compiler infrastructures, showing why the Probus project needs more than a single-language code runner and why a bytecode-like approach would be too restrictive for the intended use case.

Second, it defined DGIR as a structured IR that combines Bril’s simplicity with selected MLIR concepts such as explicit regions, operations, types and dialect-style extensibility, while deliberately omitting higher-complexity features that are not needed in the educational setting. The most central design choice shaping DGIR’s workflow was to not rely on an external declaration language like table-gen, therefore requiring a more explicit and compact API surface than MLIR.

Third, it implemented the relevant core workflow pieces: a Java compiler frontend that targets DGIR, a DGIR interpreter, and the serialization, validation and debugging support needed to connect the IR to the wider project tooling. The Java compiler frontend targeted dungeon-specific operations such as character movement, providing an integration task demonstrating the practicability of DGIR for use in domain-specific applications. Similarly, a dungeon-specific interpreter extension provided the necessary context to ensure DGIR’s interpreter can be used effectively in applications.

Fourth, it evaluated the result qualitatively against the methodological dimensions from Section 5.2, showing that the final design is simpler to work with than MLIR, more expressive than Bril, and sufficiently robust for real integration work.

### 8.1 Evaluation Summary

The evaluation shows that the thesis goal was met in the intended scope. DGIR provides a consistent representation that remains readable in serialized textual form, supports extension through clearly defined descriptors and operation structures, and can be used effectively in both a compiler frontend and an interpreter workflow.

In particular, the artifact confirms the following outcomes:

- The resulting IR is expressive enough to model the operations and control flow required by the Probus project.
- The extension model is significantly simpler than MLIR’s full feature set, while still remaining structured enough for practical use.
- The explicit source mapping and serialization rules make the IR suitable for debugging and educational inspection.
- The chosen non-SSA value model reduces authoring complexity, even though it limits some optimization-oriented workflows.

The discussion chapter also makes clear that the final design is intentionally not the most powerful compiler infrastructure possible. The current system does not aim to replace MLIR or LLVM, and it does not attempt to provide the same optimization ecosystem or runtime polymorphism facilities.

The main remaining limitations are the following:

- Advanced rewrite and optimization workflows are still limited compared to industrial compiler infrastructures.
- Optional SSA-style workflows would require additional representation work, especially around values crossing region boundaries.
- Serialization, parsing and semantic validation still require more manual implementation effort than a table-driven workflow would provide.
- The current design is intentionally scoped to the educational and prototyping context of Probus, so broader generalization must be treated cautiously.

Taken together, the results show that a practical middle ground between Bril and MLIR is achievable. The thesis therefore contributes a concrete IR design that is small enough to be approachable, structured enough to be useful, and flexible enough to serve as a foundation for multiple language frontends in an educational environment.

The next chapter expands on the remaining opportunities for development and the architectural directions that are most promising for the continued evolution of DGIR.

## 9 Future Work

This section explores potential enhancements and research areas to expand DGIR beyond its current scope and capabilities.

### 9.1 Transformations and Optimizations

While DGIR supports the same multi-level abstractions as MLIR, the IR currently lacks the tools to write the required transformations efficiently. This limits DGIR’s use as a general purpose multi-level IR. Supporting a system similar to MLIR’s pattern-rewrite mechanism and its pass infrastructure would greatly enhance DGIR optimization capabilities. But more fundamentally than that, DGIR operations, blocks, and regions lack some of the more basic functionality required to transform elements in place. While adding and removing operations or blocks is possible, creating deep copies, or replacing elements is not natively supported.

One major limiting factor to more generalized transformation capabilities is that DGIR does not use SSA form. This makes global transformations more complex, since data-flow is less clearly defined without clear def-use chains.

### 9.2 SSA Support

As discussed above, not adapting SSA-form restricts the amount of analysis and transformations that can be applied to DGIR significantly. Therefore, adding SSA support to DGIR would benefit generalized use-cases greatly. Similarly to Bril, DGIR could introduce operations providing the necessary SSA functionality through a SSA dialect. Provided that the whole program follows SSA-form, more advanced rewrite and optimization features could be enabled. This would provide the necessary guarantees making DGIR usable in more generalized contexts.

A potential syntax of such an extension could look like this:

```
1 %a: int = arith.const [value = {5}]
2 ssa.set %a [target = {%c}]
3 scf.if %cond
4 {
5   %b: int = arith.const [value = {7}] acronym
6   ssa.set %b [target = {%c}]
7 }
8 %c: int = ssa.get
9 io.print %c
```

Listing 50: Potential SSA syntax for DGIR adapted from Bril (Listing 49).

One advantage of using this syntax is that during execution, `ssa.get` instructions could be skipped completely as long as the interpreter allows defining values outside the current scope. Knowing which values are going to be used outside their definition scope has the additional advantage of being more memory friendly, as scoped values do not have to be kept alive in case a  $\phi$ -node has to access them after branching. Furthermore, using an operation to set the value removes the need to keep track of control flow and makes the region model work with minimal extensions, as control flow out of regions requires no special handling. It also enables operations with regions to define multiple SSA values without the owning operations having to define multiple return values, therefore leaving the operation structure unchanged. To fully support operations with multiple regions (like `IfOp` with `if` and `else`) additional rules would need to be introduced to ensure SSA values are always well defined. Possible restrictions would be to require SSA values to have one dominating setter in the same region, in case any setters exist in nested regions. This would ensure that every SSA value is well-defined, even if none of the defining nested regions are reached and therefore no value is set.

### 9.3 Runtime Polymorphism

Another area that limits DGIR’s use as a multi-level IR is its lack of runtime polymorphism. Without it, dialect authors cannot provide additional generalized abstractions and apply them to existing operations as needed. This means that code targeting a specific interface would only be applicable to operations implementing that interface at compile time, which may be impractical. For this reason MLIR provides a concept-model abstraction system, where the concepts declare the methods implemented by the model, and operations form a type of model. Using this system authors can implement external models wrapping an existing operation, providing the interface implementation for that operation at runtime. Adopting some form of runtime polymorphism would facilitate DGIR’s adoption as a general purpose, multi-level IR significantly.

## 9.4 Textual Support

Currently, DGIR only allows deserialization of its JSON format, with its textual representation lacking crucial information about attribute type and storage, making full deserialization impossible. Furthermore, the serialization behavior of attribute storage is not well-defined, relying on simple string conversion for the time being.

However, handwriting DGIR's textual form is a lot more user-friendly compared to writing JSON, as well as more readable and compact. Therefore, it might be beneficial to extend the attribute API to incorporate explicit storage serialization and deserialization logic, and to include the specific attribute type as part of the textual representation. This way, DGIR's textual representation could serve as a real alternative to the JSON IR form.

## 9.5 Automated Implementations

Previously, we established that one area in which DGIR's workflow falls significantly short of MLIR's table-gen-based approach, is its type parsing and printing and the validation of operation parameters. To resolve this shortcoming, DGIR could be extended to provide annotation processors, automating the generation of serialization and validation code from simple descriptors, similar to MLIR's declarative table-gen syntax.

Another, simpler approach would be to use a builder pattern to construct layout descriptors instead. These could be used to provide automated validation of structural properties and provide automated deserialization and serialization support.

A potential use of such a builder pattern might look like this:

```
1 public static final TypeSyntax syntax =
2     new TypeSyntaxBuilder()
3         .addTypeParameter(type.getElementType())
4         .addOptionalInteger(type.getWidth())
5         .build();
6
7 public String getParameterizedIdent() {
8     return syntax.buildIdent(getDetails(), getElementType(), getWidth());
9 }
10
11 public Function<Pair<String, TypeDetails>, Type> getParameterizedIdentFactory() {
12     return args -> {
13         Object[] parameters = syntax.parseIdent(args);
14         return ArrayT.of((Type) parameters[0], (OptionalInt) parameters[1]);
15     };
16 }
```

Listing 51: Possible implementation of array type serialization using builder pattern, simplifying Listing 52

```
1 public String getParameterizedIdent() {
2     return Type.buildParameterizedIdent(
3         getDetails(),
4         DgirCoreUtils.listOf(getElementType(), getWidth().isPresent() ? width : null));
5 }
6 public Function<Pair<String, TypeDetails>, Type> getParameterizedIdentFactory() {
7     return args -> {
8         AtomicReference<Type> elementType = new AtomicReference<>();
9         AtomicReference<OptionalInt> width = new AtomicReference<>();
10         Type.consumeParametricIdent(
11             args.getLeft(),
12             parameters -> {
13                 if (parameters.isEmpty() || parameters.size() > 2) {
14                     return Optional.of("Invalid number of parameters for array type: expected 1 or 2, got " + parameters.size());
15                 }
16                 if (!(parameters.getFirst() instanceof Type type))
17                     return Optional.of("Invalid parameter type for array element type: expected a type, got " +
18                                     parameters.getFirst().getClass().getSimpleName());
19                 elementType.set(type);
20                 if (parameters.size() == 2 && !(parameters.get(1) instanceof Integer))
21                     return Optional.of("Invalid parameter type for array width: expected an integer, got " +
22                                     parameters.get(1).getClass().getSimpleName());
23                 width.set(
24                     parameters.size() == 2 ? OptionalInt.of((Integer) parameters.get(1)) : OptionalInt.empty());
25                 return Optional.empty();
26             });
27         return ArrayT.of(elementType.get(), width.get());
28     };
29 }
```

Listing 52: Current implementation of array type serialization producing outputs like *mem.array<int32, 4>*.

## 9.6 Type System

One of the greatest limitations of DGIR is its rigid type system, which is not expressive enough to host the type systems of DSLs, instead relying on dialects to provide their own. In practice this means that the operations of many dialects cannot be used on types described by other dialects even though they might fulfil the semantic requirements of operands to the operation. Relying on users to implement the full type system for every dialect is not only prone to bugs, but also reduces the flexibility of IR as a whole.

Addressing these limitations Leissa et al. developed *MimIR*[17], which is a form of lambda calculus, specifically a pure type system. It is a higher-order IR allowing developers to declare so called *axioms*, which can represent any entity with a possibly polymorphic, polytypic, or dependent type. Instead of providing extension definitions externally, MimIR requires entities to be defined in *Mim* the textual representation of the IR. C++ is only used to provide transformations of the IR, while MimIR's type system handles type checks and inference automatically from the entity descriptions.

Evolving DGIR's type system into one that has a better defined notion of well-typedness, could be of significant benefit. However, redesigning DGIR as a form of lambda calculus, or even a pure type system would go against its current design goal, since such an IR would significantly raise the barrier to entry.

## References

- [1] Harold Abelson, Nat Goodman, and Lee Rudolph. “Logo manual”. In: (1974).
- [2] V Alfred et al. *Compilers Principles, Techniques*. Pearson, 2007.
- [3] Hussein Alrubaye, Stephanie Ludi, and Mohamed Wiem Mkaouer. “Comparison of block-based and hybrid-based environments in transferring programming skills to text-based environments”. In: *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering*. CASCON ’19. Toronto, Ontario, Canada: IBM Corp., 2019, pp. 100–109.
- [4] Laura Amaya, Mina Guirguis, and Joca Marquez. “Redesigning Computer Science: An Early Intervention to Combat Undergraduate Student Misconceptions”. In: (June 2019).
- [5] Peter Brusilovsky et al. “Mini-languages: a way to learn programming principles”. In: *Education and information technologies* 2.1 (1997), pp. 65–83.
- [6] Cliff Click and Michael Paleczny. “A simple graph-based intermediate representation”. In: *Papers from the 1995 ACM SIGPLAN Workshop on Intermediate Representations*. IR ’95. San Francisco, California, USA: Association for Computing Machinery, 1995, pp. 35–49. ISBN: 0897917545. DOI: 10.1145/202529.202534. URL: <https://doi.org/10.1145/202529.202534>.
- [7] Ron Cytron et al. “Efficiently computing static single assignment form and the control dependence graph”. In: *ACM Transactions on Programming Languages and Systems (TOPLAS)* 13.4 (1991), pp. 451–490.
- [8] Ulli Freiburger. *Robot Karol 3.0: Eine Programmiersprache für Schülerinnen und Schüler*. Version 3.0; accessed 2025-12-16. mebis – Landesmedienzentrum Bayern. 2015. URL: <https://mebis.bycs.de/cdn/assets/dsc/posts/15ab2c3c-3dd3-4630-bc56-4baba4252cdc/2-2015-05-karol30handbuch.pdf>.
- [9] Google Developers. *Blockly*. accessed 2025-12-16. 2025. URL: <https://developers.google.com/blockly>.
- [10] Google Developers. *What Is Blockly?* accessed 2025-12-16. 2025. URL: <https://developers.google.com/blockly/guides/get-started/what-is-blockly>.
- [11] Initiative D21 e. V. *D21-Digital-Index 2024/25: Jährliches Lagebild zur Digitalen Gesellschaft*. Technical Report. Durchgeführt von Kantar; repräsentative Studie zur Digitalisierung in Deutschland. Initiative D21, 2025. URL: [https://initiatived21.de/uploads/03\\_Studien-Publikationen/D21-Digital-Index/2024-25/D21DigitalIndex\\_2024-2025.pdf](https://initiatived21.de/uploads/03_Studien-Publikationen/D21-Digital-Index/2024-25/D21DigitalIndex_2024-2025.pdf).
- [12] JetBrains. *Kotlin Programming Language*. Accessed: 2026-04-09. 2025. URL: <https://kotlinlang.org/>.
- [13] Steve Klabnik and Carol Nichols. *The Rust Programming Language*. Accessed: 2026-04-09. No Starch Press, 2019. URL: <https://doc.rust-lang.org/book/>.
- [14] Chris Lattner. “LLVM and Clang: Next Generation Compiler Technology”. In: *The BSD Conference*. 2008.
- [15] Chris Lattner and Vikram Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation”. In: *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. 2004, pp. 75–86.
- [16] Chris Lattner et al. “MLIR: A compiler infrastructure for the end of Moore’s law”. In: *arXiv preprint arXiv:2002.11054* (2020).
- [17] Roland Leißa et al. “MimIR: An Extensible and Type-Safe Intermediate Representation for the DSL Age”. In: *Proc. ACM Program. Lang.* 9.POPL (Jan. 2025). DOI: 10.1145/3704840. URL: <https://doi.org/10.1145/3704840>.
- [18] Quinn Li. *Robot Karol Online*. Accessed: 2026-04-12. 2026. URL: <https://karol.arrrg.de/>.
- [19] LLVM Project. *MLIR: Multi-Level Intermediate Representation*. <https://mlir.llvm.org/>. Accessed: 2026-04-10.
- [20] Bruno Cardoso Lopes. “Introducing ClangIR: An MLIR-Based High-Level IR for the C/C++ Family of Languages”. In: *Proceedings of the Tenth Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC2024)*. Co-located with SC24: The International Conference for High Performance Computing, Networking, Storage, and Analysis. Atlanta, GA, USA, Nov. 2024. URL: [https://sc24.supercomputing.org/proceedings/workshops/workshop\\_pages/misc312.html](https://sc24.supercomputing.org/proceedings/workshops/workshop_pages/misc312.html).
- [21] André Matutat et al. “Designing Workshops to Bridge Visual and Textual Programming: A Narrative Game Approach”. In: *Proceedings of the 10th Annual International GamiFIN Conference 2026*. GamiFIN ’26. Association for Computing Machinery, 2026. ISBN: 9798400721908. DOI: 10.1145/3789624.3789625. URL: <https://doi.org/10.1145/3789624.3789625>.

- [22] MIT Scratch Team. *About Scratch*. accessed 2025-12-16. 2025. URL: <https://scratch.mit.edu/about>.
- [23] MIT Scratch Team. *Scratch Credits*. accessed 2025-12-16. 2025. URL: <https://scratch.mit.edu/credits>.
- [24] Richard E Pattis. *Karel the robot: a gentle introduction to the art of programming*. John Wiley & Sons, 1994.
- [25] Mitchel Resnick et al. “Scratch: programming for all”. In: *Commun. ACM* 52.11 (Nov. 2009), pp. 60–67. ISSN: 0001-0782. DOI: 10.1145/1592761.1592779. URL: <https://doi.org/10.1145/1592761.1592779>.
- [26] Alexander Ruf, Andreas Mühling, and Peter Hubwieser. “Scratch vs. Karel: Impact on learning outcomes and motivation”. In: *Proceedings of the 9th workshop in primary and secondary computing education*. 2014, pp. 50–59.
- [27] Adrian Sampson. *Bril: A Compiler Intermediate Representation for Learning*. <https://github.com/sampsyo/bril>. Accessed: 2026-04-12.
- [28] Adrian Sampson. *Bril: A Compiler Intermediate Representation for Learning — Language Reference and Tool Documentation*. Accessed: 2026-04-12.
- [29] Adrian Sampson. *Bril: An Intermediate Language for Teaching Compilers*. <https://www.cs.cornell.edu/~asampson/blog/bril.html>. Accessed: 2026-04-12. July 2024.
- [30] Cynthia Solomon et al. “History of logo”. In: *Proceedings of the ACM on Programming Languages* 4.HOPL (2020), pp. 1–66.
- [31] James Stanier and Des Watson. “Intermediate representations in imperative compilers: A survey”. In: *ACM Computing Surveys (CSUR)* 45.3 (2013), pp. 1–27.

## A Appendix

```
1 <program>      ::= <operation>
2
3 <operation>    ::= <output_decl>? <ident> <operands>? <attributes>? <dyn_attributes>? <successors>? <location>? <region>*
4
5 <output_decl>  ::= <typed_value> '='
6
7 <operands>     ::= <value> ( ',' <value> )*
8
9 <attributes>   ::= '[' <attribute_list> ']'
10 <attribute_list> ::= <attribute> ( ',' <attribute> )*
11 <attribute>    ::= <identifier> '=' '{' <attr_value> '}'
12 <attr_value>   ::= <attr_char>*
13 <attr_char>    ::= '{' <attr_char>* '}' | <string_literal> | <any_char_except_brace_or_quote>
14
15 <dyn_attributes> ::= '<dynamic>' '[' <attribute_list> ']'
16
17 <successors>   ::= '==>' '[' <block_name> ( ',' <block_name> )* ']'
18
19 <region>       ::= <value_decl_list>? '{' <block>+ '}'
20 <value_decl_list> ::= '(' <typed_value> ( ',' <typed_value> )* ')'
21
22 <block>        ::= <block_name> ':' <operation>+
23 <block_name>   ::= '.blk_' <number>
24
25 <typed_value>  ::= <value> ':' <type>
26 <value>        ::= '%' <value_ident>
27 <value_ident>  ::= <identifier> ( '_' <number> )? | <number>
28
29 <location>     ::= '@' <location_source> ':' <number> ':' <number>
30 <location_source> ::= '<' <ident> '>' | <ident>
31
32 <type>         ::= <ident> <type_params>?
33 <type_params>  ::= '<' <type_param> ( ',' <type_param> )* '>'
34 <type_param>   ::= <type> | <string_literal> | <number>
35
36 <string_literal> ::= '"' <any_char>* '"'
37
38 <ident>        ::= <identifier> ( '.' <identifier> )*
39
40 <identifier>   ::= <letter> ( <letter> | <digit> | '_' )*
41 <number>       ::= '-'? <digit>+
42 <letter>       ::= 'a' | ... | 'z' | 'A' | ... | 'Z'
43 <digit>        ::= '0' | ... | '9'
```

Listing 53: Approximate BNF grammar of textual DGIR programs.

```

1 {
2   "functions": [
3     {
4       "instrs": [
5         {
6           "dest": "a",
7           "op": "const",
8           "type": "int",
9           "value": 47
10        },
11        {
12          "dest": "b",
13          "op": "const",
14          "type": "int",
15          "value": 42
16        },
17        {
18          "dest": "cond",
19          "op": "const",
20          "type": "bool",
21          "value": true
22        },
23        {
24          "args": [
25            "cond"
26          ],
27          "labels": [
28            "left",
29            "right"
30          ],
31          "op": "br"
32        },
33        {
34          "label": "left"
35        },
36        {
37          "dest": "b",
38          "op": "const",
39          "type": "int",
40          "value": 1
41        },
42        {
43          "dest": "c",
44          "op": "const",
45          "type": "int",
46          "value": 5
47        },
48        {
49          "labels": [
50            "end"
51          ],
52          "op": "jmp"
53        },
54        "...
55      ],
56      "name": "main"
57    }
58  ]
59 }

```

Listing 54: Bril JSON IR form of Listing 46 (shortened).

```

1 {
2   "ident": "program",
3   "regions": [ {
4     "blocks": [ {
5       "id": ".blk_0",
6       "operations": [ {
7         "ident": "func.func",
8         "attributes": [ {
9           "name": "symbol_name",
10          "attr_ident": "stringAttr",
11          "attr_type": "string",
12          "attr_value": "main"
13        } ],
14        "name": "type",
15        "attr_ident": "typeAttr",
16        "attr_type": "func.func<\"() -> ()\">"
17      } ],
18      "regions": [ {
19        "blocks": [ {
20          "id": ".blk_1",
21          "operations": [ {
22            "ident": "arith.constant",
23            "attributes": [ {
24              "name": "value",
25              "attr_ident": "integerAttr",
26              "attr_type": "int64",
27              "attr_value": 47
28            } ],
29            "output": {
30              "id": "%0",
31              "type": "int64"
32            }
33          }, {
34            "ident": "arith.constant",
35            "attributes": [ {
36              "name": "value",
37              "attr_ident": "integerAttr",
38              "attr_type": "int64",
39              "attr_value": 42
40            } ],
41            "output": {
42              "id": "%1",
43              "type": "int64"
44            }
45          }, {
46            "ident": "arith.constant",
47            "attributes": [ {
48              "name": "value",
49              "attr_ident": "integerAttr",
50              "attr_type": "int1",
51              "attr_value": true
52            } ],
53            "output": {
54              "id": "%2",
55              "type": "int1"
56            }
57          }, {
58            "ident": "cf.br_cond",
59            "operands": [ "%2" ],
60            "successors": [ ".blk_2", ".blk_3" ]
61          } ]
62        }, {
63          "id": ".blk_2",
64          "operations": [ {
65            "ident": "arith.constant",
66            "attributes": [ {
67              "name": "value",
68              "attr_ident": "integerAttr",
69              "attr_type": "int64",
70              "attr_value": 1
71            } ],
72            "output": "%1"
73          }, {
74            "ident": "arith.constant",
75            "attributes": [ {
76              "name": "value",
77              "attr_ident": "integerAttr",
78              "attr_type": "int64",
79              "attr_value": 5
80            } ],
81            "output": {
82              "id": "%3",
83              "type": "int64"
84            }
85          }, {
86            "ident": "cf.br",
87            "successors": [ ".blk_4" ]
88          } ]
89        }, "...
90      ]
91    } ]
92  ]
93 } ]
94 } ]
95 }

```

Listing 55: DGIR JSON IR form of Listing 46 (shortened).

```

1 # Entry: validate all top-level regions of operation `root`
2 function validate(root):
3     problems = []
4     for region in root.regions:
5         seed = seedForRegion(root, region, Set<Value>())
6         analyzeRegion(region, seed, problems)
7     return problems
8
9 # Collect values visible to blocks inside child region of op
10 function seedForRegion(op, region, parentState):
11     seed = {region.values}
12     return op.isolatedFromAbove ? seed : (seed + parentState)
13
14 # Analyze one region with incoming seed values
15 function analyzeRegion(region, incoming, problems):
16     preds = map each region.blocks -> block.predecessors
17     in = map<Block, Set<Value>>()
18     out = map<Block, Set<Value>>()
19
20     # Phase 1: Top level analysis
21     worklist = all region.blocks
22     while worklist non-empty:
23         b = worklist.pop()
24         in_b = if preds[b].empty:
25             (incoming if b is entry else {})
26         else
27             intersection( out[p] for p in preds[b] )
28         out_b = in_b ? GEN(b) # GEN(b) = values defined by ops in b
29         if in_b != in[b] or out_b != out[b]:
30             in[b]=in_b; out[b]=out_b
31             for s in b.successors: worklist.push(s)
32
33     # Phase 2: intra-block scan + recursion into child regions
34     for b in region.blocks:
35         state = in.getOrDefault(b, incoming) # reaching defs at block entry
36         for op in b.operations:
37             for operand in op.operands:
38                 if operand.value not in state:
39                     problems.add(missing-def diagnostic)
40             # recurse into nested regions
41             for child in op.regions:
42                 childSeed = seedForRegion(op, child, state)
43                 analyzeRegion(child, childSeed, problems)
44             # transfer: op's output becomes available after op executes
45             if op.hasOutput: state.add(op.output)

```

Listing 56: Iterative reaching-definitions algorithm used by DGIR during static analysis.

```

1 public static void main() {
2     int x = 2;
3     String result = "";
4     switch(x) {
5         case 1:
6             result = "One";
7             break;
8         case 2:
9             result = "Two";
10            break;
11        case 3:
12            result = "Three";
13            break;
14        default:
15            result = "Other";
16    }
17 }

```

Listing 57: Basic switch statement in Java.

```

1 func.func [symbol_name = {main}, type = {func.func<"() -> ()">}] @simpleSwitchStatement.java:2:3
2 {
3     .blk_0:
4     %x_0 :int32 = arith.constant [value = {2}] @simpleSwitchStatement.java:3:13
5     %result_1 :string = arith.constant [value = {}] @simpleSwitchStatement.java:4:21
6     scf.scope @<ignore>:-1:-1
7     {
8         .blk_1:
9         %2 :int32 = arith.constant [value = {1}] @simpleSwitchStatement.java:6:12
10        %3 :int1 = arith.bin %x_0, %2 [binMode = {EQ}] @<ignore>:-1:-1
11        scf.if %3 @<ignore>:-1:-1
12        {
13            .blk_2:
14            %result_1 :string = arith.constant [value = {One}] @simpleSwitchStatement.java:7:18
15            scf.end @<ignore>:-1:-1
16        }
17        {
18            .blk_3:
19            scf.scope @<ignore>:-1:-1
20            {
21                .blk_4:
22                %4 :int32 = arith.constant [value = {2}] @simpleSwitchStatement.java:9:12
23                %5 :int1 = arith.bin %x_0, %4 [binMode = {EQ}] @<ignore>:-1:-1
24                scf.if %5 @<ignore>:-1:-1
25                {
26                    .blk_5:
27                    %result_1 :string = arith.constant [value = {Two}] @simpleSwitchStatement.java:10:18
28                    scf.end @<ignore>:-1:-1
29                }
30                {
31                    .blk_6:
32                    scf.scope @<ignore>:-1:-1
33                    {
34                        .blk_7:
35                        %6 :int32 = arith.constant [value = {3}] @simpleSwitchStatement.java:12:12
36                        %7 :int1 = arith.bin %x_0, %6 [binMode = {EQ}] @<ignore>:-1:-1
37                        scf.if %7 @<ignore>:-1:-1
38                        {
39                            .blk_8:
40                            %result_1 :string = arith.constant [value = {Three}] @simpleSwitchStatement.java:13:18
41                            scf.end @<ignore>:-1:-1
42                        }
43                        {
44                            .blk_9:
45                            %result_1 :string = arith.constant [value = {Other}] @simpleSwitchStatement.java:16:18
46                            scf.end @<ignore>:-1:-1
47                        }
48                        scf.end @<ignore>:-1:-1
49                    }
50                    scf.end @<ignore>:-1:-1
51                }
52                scf.end @<ignore>:-1:-1
53            }
54            scf.end @<ignore>:-1:-1
55        }
56        scf.end @<ignore>:-1:-1
57    }
58 }

```

Listing 58: Switch statement compiled to DGIR structured control flow.

```

1 final class FunctionDescriptor implements FuncTypeDescriptor {
2     @Override
3     Class<? extends Dialect> getDialect() { return FuncDialect.class; }
4     public static List<TypeDescriptor> getDescriptors() { return List.of(new FunctionDescriptor()); }
5     @Override
6     public Class<? extends Type> getTypeClass() { return FuncType.class; }
7     @Override
8     public String getIdent() { return "func.func"; }
9     @Override
10    public Function<Object, Boolean> getValidator() { return value -> true; }
11    @Override
12    public void initDefaultTypeInstances() {}
13    @Override
14    public Function<Pair<String, TypeDetails>, Type>
15        getParameterizedIdentFactory() {
16        return args -> {
17            Matcher matcher = FUNC_TYPE_PATTERN.matcher(args.getLeft());
18            if (!matcher.matches()) {
19                throw new IllegalArgumentException(
20                    "Invalid parameterized ident for func.func type: %s".formatted(args.getLeft()));
21            }
22            // Everything inside first ()
23            String inputsPart = Type.unescapeCustomExpression(matcher.group(1));
24            List<Type> inputs = new ArrayList<>();
25            Type.consumeParameterText(inputsPart, Type.AllTypes.of(inputs));
26
27            // Everything inside second ()
28            String outputPart = Type.unescapeCustomExpression(matcher.group(2));
29            Type output = null;
30            if (!outputPart.isBlank()) output = Type.fromParameterizedIdent(outputPart);
31
32            return FuncType.of(inputs, output);
33        };
34    }
35    private static final Pattern FUNC_TYPE_PATTERN =
36        Pattern.compile("^func\\.func<\"\\((.*)\\)\\s*->\\s*\"\\((.*)\\)>$");
37 }
38
39 class FuncType extends Type {
40     @Override
41     public String getParameterizedIdent() {
42         String inputs = Type.buildParameterList(getInputs());
43         String output = getOutput() != null ? getOutput().toString() : "";
44         String expression = "(%s) -> (%s)".formatted(inputs, output);
45         return Type.buildParameterizedIdent(getDetails(), List.of(expression));
46     }
47     public static FuncType empty() {
48         return TypeUniquer.uniqueInstance(new FuncType());
49     }
50     public static FuncType of(List<Type> inputs, Type output) {
51         return TypeUniquer.uniqueInstance(new FuncType(inputs, output));
52     }
53
54     private final List<Type> inputs;
55     private final Type output;
56
57     private FuncType() {
58         super("func.func");
59         inputs = List.of();
60         output = null;
61     }
62     private FuncType(List<Type> inputs, Type output) {
63         super("func.func");
64         this.inputs = Collections.unmodifiableList(inputs);
65         this.output = output;
66     }
67     public List<Type> getInputs() { return inputs; }
68     public Type getOutput() { return output; }
69 }

```

Listing 59: Implementation of function type in DGIR.

```

1 def Arith_ConstantOp : Op<Arith_Dialect, "constant",
2   [ConstantLike, Pure,
3     DeclareOpInterfaceMethods<OpAsmOpInterface, ["getAsmResultNames"]>,
4     AllTypesMatch<["value", "result"]>,
5     DeclareOpInterfaceMethods<InferIntRangeInterface, ["inferResultRanges"]>> {
6   let summary = "integer or floating point constant";
7   let description = [{
8     The constant operation produces an SSA value equal to some integer or
9     floating-point constant specified by an attribute. This is the way MLIR
10    forms simple integer and floating point constants.
11
12    Example:
13
14    
15    // Integer constant
16    %1 = arith.constant 42 : i32
17
18    // Equivalent generic form
19    %1 = "arith.constant"() {value = 42 : i32} : () -> i32
20    
21  ]};
22
23  let arguments = (ins TypedAttrInterface:$value);
24  let results = (outs /*SignlessIntegerOrIndexOrFloatLike*/AnyType:$result);
25
26  let extraClassDeclaration = [{
27    static bool isBuildableWith(Attribute value, Type type);
28
29    static ConstantOp materialize(OpBuilder &builder, Attribute value,
30                                  Type type, Location loc);
31  }];
32
33  let hasFolder = 1;
34  let assemblyFormat = "attr-dict $value";
35  let hasVerifier = 1;
36 }

```

Listing 60: Table-gen declaration of MLIR’s ConstantOp.

```

1 class ConstantOp : public ::mlir::Op<ConstantOp, ::mlir::OpTrait::ZeroRegions, ::mlir::OpTrait::OneResult, ::mlir::OpTrait::OneTypedResult<::mlir::Type>::Impl,
  ↳ ::mlir::OpTrait::ZeroSuccessors, ::mlir::OpTrait::ZeroOperands, ::mlir::OpTrait::OpInvariants, ::mlir::BytecodeOpInterface::Trait, ::mlir::OpTrait::ConstantLike,
  ↳ ::mlir::ConditionallySpeculatable::Trait, ::mlir::OpTrait::AlwaysSpeculatableImplTrait, ::mlir::MemoryEffectOpInterface::Trait, ::mlir::OpAsmOpInterface::Trait,
  ↳ ::mlir::InferIntRangeInterface::Trait, ::mlir::InferTypeOpInterface::Trait> {
2 public:
3   using Op::Op;
4   using Op::print;
5   using Adaptor = ConstantOpAdaptor;
6   template <typename RangeT>
7   using GenericAdaptor = ConstantOpGenericAdaptor<RangeT>;
8   using FoldAdaptor = GenericAdaptor<::llvm::ArrayRef<::mlir::Attribute>>;
9   using Properties = FoldAdaptor::Properties;
10  using ::mlir::OpAsmOpInterface::Trait<ConstantOp>::getAsmBlockArgumentNames;
11  using ::mlir::OpAsmOpInterface::Trait<ConstantOp>::getAsmBlockNames;
12  using ::mlir::OpAsmOpInterface::Trait<ConstantOp>::getDefaultDialect;
13  using ::mlir::InferIntRangeInterface::Trait<ConstantOp>::inferResultRangesFromOptional;
14  static ::llvm::ArrayRef<::llvm::StringRef> getAttributeNames() {
15    static ::llvm::StringRef attrNames[] = {::llvm::StringRef("value")};
16    return ::llvm::ArrayRef(attrNames);
17  }
18  ::mlir::StringRef getValueAttrName() {
19    return getAttributeNameForIndex(0);
20  }
21  static ::mlir::StringRef getValueAttrName(::mlir::OperationName name) {
22    return getAttributeNameForIndex(name, 0);
23  }
24  static constexpr ::llvm::StringLiteral getOperationName() {
25    return ::llvm::StringLiteral("arith.constant");
26  }
27  std::pair<unsigned, unsigned> getODSOperandIndexAndLength(unsigned index) {
28    return {index, 1};
29  }
30  ::mlir::Operation::operand_range getODSOperands(unsigned index) {
31    auto valueRange = getODSOperandIndexAndLength(index);
32    return {std::next(getOperation()->operand_begin(), valueRange.first),
33            std::next(getOperation()->operand_begin(), valueRange.first + valueRange.second)};
34  }
35  std::pair<unsigned, unsigned> getODSResultIndexAndLength(unsigned index) {
36    return {index, 1};
37  }
38  ::mlir::Operation::result_range getODSResults(unsigned index) {
39    auto valueRange = getODSResultIndexAndLength(index);
40    return {std::next(getOperation()->result_begin(), valueRange.first),
41            std::next(getOperation()->result_begin(), valueRange.first + valueRange.second)};
42  }
43  ::mlir::TypedValue<::mlir::Type> getResult() {
44    return ::llvm::cast<::mlir::TypedValue<::mlir::Type>>(&getODSResults(0).begin());
45  }
46  static ::llvm::LogicalResult setPropertiesFromAttr(Properties &prop, ::mlir::Attribute attr, ::llvm::function_ref<::mlir::InFlightDiagnostic()> emitError);
47  static ::mlir::Attribute getPropertiesAsAttr(::mlir::MLIRContext *ctx, const Properties &prop);
48  static llvm::hash_code computePropertiesHash(const Properties &prop);
49  static std::optional<::mlir::Attribute> getInherentAttr(::mlir::MLIRContext *ctx, const Properties &prop, llvm::StringRef name);
50  static void setInherentAttr(Properties &prop, llvm::StringRef name, ::mlir::Attribute value);
51  static void populateInherentAttrs(::mlir::MLIRContext *ctx, const Properties &prop, ::mlir::NamedAttrList &attrs);
52  static ::llvm::LogicalResult verifyInherentAttrs(::mlir::OperationName opName, ::mlir::NamedAttrList &attrs, llvm::function_ref<::mlir::InFlightDiagnostic()> emitError);
53  static ::llvm::LogicalResult readProperties(::mlir::DialectBytecodeReader &reader, ::mlir::OperationState &state);
54  void writeProperties(::mlir::DialectBytecodeWriter &writer);
55  ::mlir::TypedAttr getValueAttr() {
56    return ::llvm::cast<::mlir::TypedAttr>(getProperties().value);
57  }
58  ::mlir::TypedAttr getValue();
59  void setValueAttr(::mlir::TypedAttr attr) {
60    getProperties().value = attr;
61  }
62  static void build(::mlir::OpBuilder &builder, ::mlir::OperationState &state, ::mlir::Type result, ::mlir::TypedAttr value);
63  static ConstantOp create(::mlir::OpBuilder &builder, ::mlir::Location location, ::mlir::Type result, ::mlir::TypedAttr value);
64  static ConstantOp create(::mlir::ImplicitLocOpBuilder &builder, ::mlir::Type result, ::mlir::TypedAttr value);
65  static void build(::mlir::OpBuilder &builder, ::mlir::OperationState &state, ::mlir::TypedAttr value);
66  static ConstantOp create(::mlir::OpBuilder &builder, ::mlir::Location location, ::mlir::TypedAttr value);
67  static ConstantOp create(::mlir::ImplicitLocOpBuilder &builder, ::mlir::TypedAttr value);
68  static void build(::mlir::OpBuilder &builder, ::mlir::OperationState &state, ::mlir::TypeRange resultTypes, ::mlir::TypedAttr value);
69  static ConstantOp create(::mlir::OpBuilder &builder, ::mlir::Location location, ::mlir::TypeRange resultTypes, ::mlir::TypedAttr value);
70  static ConstantOp create(::mlir::ImplicitLocOpBuilder &builder, ::mlir::TypeRange resultTypes, ::mlir::TypedAttr value);
71  static void build(::mlir::OpBuilder &, ::mlir::OperationState &state, ::mlir::TypeRange resultTypes, ::mlir::ValueRange operands, ::llvm::ArrayRef<::mlir::NamedAttribute>
  ↳ attributes = {});
72  static ConstantOp create(::mlir::OpBuilder &builder, ::mlir::Location location, ::mlir::TypeRange resultTypes, ::mlir::ValueRange operands,
  ↳ ::llvm::ArrayRef<::mlir::NamedAttribute> attributes = {});
73  static ConstantOp create(::mlir::ImplicitLocOpBuilder &builder, ::mlir::TypeRange resultTypes, ::mlir::ValueRange operands, ::llvm::ArrayRef<::mlir::NamedAttribute> attributes =
  ↳ {});
74  static void build(::mlir::OpBuilder &builder, ::mlir::OperationState &state, ::mlir::TypeRange resultTypes, ::mlir::ValueRange operands, ::llvm::ArrayRef<::mlir::NamedAttribute> attributes = {});
75  static ConstantOp create(::mlir::OpBuilder &builder, ::mlir::Location location, ::mlir::ValueRange operands, ::llvm::ArrayRef<::mlir::NamedAttribute> attributes = {});
76  static ConstantOp create(::mlir::ImplicitLocOpBuilder &builder, ::mlir::ValueRange operands, ::llvm::ArrayRef<::mlir::NamedAttribute> attributes = {});
77  static void build(::mlir::OpBuilder &, ::mlir::OperationState &state, ::mlir::TypeRange resultTypes, ::mlir::ValueRange operands, const Properties &properties,
  ↳ ::llvm::ArrayRef<::mlir::NamedAttribute> discardableAttributes = {});
78  static ConstantOp create(::mlir::OpBuilder &builder, ::mlir::Location location, ::mlir::TypeRange resultTypes, ::mlir::ValueRange operands, const Properties &properties,
  ↳ ::llvm::ArrayRef<::mlir::NamedAttribute> discardableAttributes = {});
79  static ConstantOp create(::mlir::ImplicitLocOpBuilder &builder, ::mlir::TypeRange resultTypes, ::mlir::ValueRange operands, const Properties &properties,
  ↳ ::llvm::ArrayRef<::mlir::NamedAttribute> discardableAttributes = {});
80  static void build(::mlir::OpBuilder &builder, ::mlir::OperationState &state, ::mlir::ValueRange operands, const Properties &properties,
  ↳ ::llvm::ArrayRef<::mlir::NamedAttribute> discardableAttributes = {});
81  static ConstantOp create(::mlir::OpBuilder &builder, ::mlir::Location location, ::mlir::ValueRange operands, const Properties &properties,
  ↳ ::llvm::ArrayRef<::mlir::NamedAttribute> discardableAttributes = {});
82  static ConstantOp create(::mlir::ImplicitLocOpBuilder &builder, ::mlir::ValueRange operands, const Properties &properties, ::llvm::ArrayRef<::mlir::NamedAttribute>
  ↳ discardableAttributes = {});
83  ::llvm::LogicalResult verifyInvariantsImpl();
84  ::llvm::LogicalResult verifyInvariants();
85  ::llvm::LogicalResult verify();
86  ::mlir::OpFoldResult fold(FoldAdaptor adaptor);
87  static ::llvm::LogicalResult inferReturnTypes(::mlir::MLIRContext *context, std::optional<::mlir::Location> location, ::mlir::ValueRange operands, ::mlir::DictionaryAttr
  ↳ attributes, ::mlir::PropertyRef properties, ::mlir::RegionRange regions, ::llvm::SmallVectorImpl<::mlir::Type>&inferredReturnTypes);
88  void getAsmResultNames(::mlir::OpAsmSetValueNameFn setNameFn);
89  void inferResultRanges(::llvm::ArrayRef<::mlir::ConstantIntRanges> argRanges, ::mlir::SetIntRangeFn setResultRanges);
90  static ::mlir::ParseResult parse(::mlir::OpAsmParser &parser, ::mlir::OperationState &result);
91  void print(::mlir::OpAsmPrinter &odsPrinter);
92  void getEffects(::llvm::SmallVectorImpl<::mlir::SideEffects::EffectInstance<::mlir::MemoryEffects::Effect>> &effects);
93 private:
94  ::mlir::StringRef getAttributeNameForIndex(unsigned index) {
95    return getAttributeNameForIndex(&this->getName(), index);
96  }
97  static ::mlir::StringRef getAttributeNameForIndex(::mlir::OperationName name, unsigned index) {
98    assert(index < 1 && "invalid attribute index");
99    assert(name.getStringRef() == getOperationName() && "invalid operation name");
100    assert(name.isRegistered() && "Operation isn't registered, missing a "
101           "dependent dialect loading?");
102    return name.getAttributeNames()[index];
103  }
104 public:
105  static bool isBuildableWith(Attribute value, Type type);
106  static ConstantOp materialize(OpBuilder &builder, Attribute value,
107                               Type type, Location loc);
108 };

```

Listing 61: MLIR's ConstantOp class declaration generated by table-gen(Listing 60).

```

1 final class ConstantOp extends ArithOp implements ArithOps, INoOperands, IHasResult {
2   @Contract(pure = true)
3   @Override
4   public String getIdent() {
5     return "arith.constant";
6   }
7
8   @Override
9   public Function<Operation, Boolean> getVerifier() {
10    return ignored -> true;
11  }
12
13  @Override
14  public Function<Operation, Op> getOpFactory() {
15    return operation -> new ConstantOp().setOperation(operation);
16  }
17
18  @Contract(pure = true)
19  @Override
20  public Supplier<List<NamedAttribute>>
21    defaultAttributes() {
22    return () -> List.of(new NamedAttribute("value"));
23  }
24
25  private ConstantOp() {}
26
27  public ConstantOp(Location location, TypedAttribute value) {
28    setOperation(Operation.Create(location, this, null, null, value.getType()));
29    getAttributesMap().get("value").setAttribute(value);
30  }
31
32  public ConstantOp(Location location, String value) { this(location, new StrAttrs.StringAttribute(value)); }
33  public ConstantOp(Location location, int value) { this(location, new IntegerAttribute(value, IntegerT.INT32())); }
34  public ConstantOp(Location location, long value) { this(location, new IntegerAttribute(value, IntegerT.INT64())); }
35  public ConstantOp(Location location, boolean value) { this(location, new IntegerAttribute(value ? 1 : 0, IntegerT.BOOL())); }
36
37  public TypedAttribute getValueAttribute() {
38    return getAttributeAs("value", TypedAttribute.class)
39      .orElseThrow(() -> new AssertionError("No value attribute found.));
40  }
41
42  public Type getValueType() {
43    return getValueAttribute().getType();
44  }
45
46  public Object getValueStorage() {
47    return getValueAttribute().getStorage();
48  }
49 }

```

Listing 62: DGIR's ConstantOp implementation.

10/2023

## I. Eigenständigkeitserklärung\*

*Declaration of originality\**

Hiermit versichere ich

*Hereby, I*

**Foster, Lasse**

Name, Vorname

*Last name, First name*

**50049857**

Matrikelnummer

*Student ID number*

**Informatik**

Studiengang

*Study programme*

dass ich die vorliegende

Masterarbeit / master thesis

*affirm that I have prepared the present*

(bei Gruppenarbeit mein bearbeiteter Teil) mit dem Thema

*(in case of group work the part I have prepared) with the topic*

Klicken oder tippen Sie hier, um Text einzugeben.

Design of an extendible Intermediate Representation for Multi-Language Game-Based Programming Education

selbstständig und ohne die Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Alle Stellen – einschließlich Tabellen, Karten, Abbildungen etc. –, die wörtlich oder sinngemäß aus veröffentlichten und nicht veröffentlichten Werken und Quellen (dazu zählen auch Internetquellen) entnommen wurden, sind in jedem einzelnen Fall mit exakter Quellenangabe kenntlich gemacht worden.

*independently and without using any other than the indicated aids. All passages – including tables, maps, figures, etc. – taken verbatim or rephrased from published and unpublished works and sources (including Internet sources) have been identified in each individual case with exact reference to the source.*

Zusätzlich versichere ich, dass ich beim Einsatz von generativen IT-/KI-Werkzeugen (z. B. ChatGPT, BARD, Dall-E oder Stable Diffusion) diese Werkzeuge in einer Rubrik „Übersicht verwendeter Hilfsmittel“ mit ihrem Produktnamen, der Zugriffsquelle (z. B. URL) und Angaben zu genutzten Funktionen der Software sowie Nutzungsumfang vollständig angeführt habe. Wörtliche sowie paraphrasierende Übernahmen aus Ergebnissen dieser Werkzeuge habe ich analog zu anderen Quellenangaben gekennzeichnet.

*In addition, I assure that, when using generative IT/AI tools (e.g., ChatGPT, BARD, Dall-E, Stable Diffusion), I have listed these tools in full in a section "Overview of tools used" with their product name, the access source (e.g., the URL) and information on the functions of the software used as well as the scope of use. I have marked verbatim and paraphrased quotes from the results of these tools in the same way as I have marked other sources.*

Mir ist bekannt, dass es sich bei einem Plagiat um eine Täuschung handelt, die gemäß der Prüfungsordnung sanktioniert werden wird.

*I am aware that plagiarism is a form of cheating that will be penalised according to the examination regulations.*

Ich versichere, dass ich die vorliegende Arbeit oder Teile daraus nicht bereits anderweitig innerhalb oder außerhalb der Hochschule als Prüfungsleistung eingereicht habe.

10/2023

*I certify that I have not already submitted the present work or parts thereof as an examination performance elsewhere within or outside the university.*

**Minden, 05.05.2026**

Ort, Datum  
Place, date

  
\_\_\_\_\_  
Unterschrift  
Signature

\* Bitte legen Sie diese Eigenständigkeitserklärung ausgefüllt und unterzeichnet Ihrer Arbeit am Ende bei. Sollte diese fehlen, wird die Arbeit nicht korrigiert bzw. bei endgültiger Nichtvorlage als Täuschungsversuch gewertet.

\* Please complete and sign this declaration of originality and enclose it with your work at the end. If this is missing, the work will not be evaluated or, in case of final non-submission, it will be considered an attempt to cheat.